



**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge–Fog–Cloud**

Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra

Universidad de Los Andes
Departamento de Ingeniería de sistemas y computación
Bogotá, Colombia
2026

**Sistema de detección de intrusiones para redes IoT
mediante TinyML, aprendizaje federado jerárquico y
cifrado ligero ASCON-128 en una arquitectura
Edge–Fog–Cloud.**

**Santiago Alejandro Jaimes Puerto
Nicolas Casas Ibarra**

University de Los Andes
Presentado en cumplimiento parcial de los requisitos para el grado de:
System and Computer engineer

Director:
Prof. Carlos Andrés Lozano Garzón, PhD
Trabajo de grado defendido en:
Mayo 24, 2026 - 12:00 pm

Universidad de Los Andes
Departamento de Ingeniería de Sistemas y Computación
Bogotá, Colombia
2026

Agradecimientos

Queremos expresar nuestra más sincera gratitud a todas las personas que hicieron posible la culminación de este trabajo de grado.

A nuestras familias, quienes han sido nuestra ancla y nuestra inspiración en cada etapa de este camino. Su amor incondicional, su paciencia infinita y su apoyo constante a lo largo de estos años nos dieron la fortaleza necesaria para superar cada desafío académico y personal. Este logro es también suyo, porque sin su confianza y su acompañamiento nada de esto habría sido posible.

Al profesor Carlos Andrés Lozano Garzón, nuestro director de tesis, le agradecemos profundamente por compartir con nosotros su conocimiento, su experiencia y su tiempo. Su aporte fue clave desde el principio: nos ayudó a delimitar y desglosar el problema cuando todavía era una idea difusa, y su forma de guiar sin imponer ya la conocíamos de materias anteriores. Sus anécdotas de clase, que en su momento parecían digresiones, terminaron formando una manera de pensar que se nota en cada decisión de este trabajo.

A la Universidad de los Andes le debemos más de lo que cabe resumir aquí. La calidad de sus profesores, la ambición que te contagia el ambiente y las amistades que deja son cosas difíciles de encontrar juntas. Y hay un lugar en particular que merece mención: el primer piso del ML, donde se pasó la mayor parte de estos semestres entre código, café y conversaciones que iban mucho más allá de los trabajos.

Nicolás quiere agradecer especialmente a Marco y a Esteban, dos de los mejores amigos que deja esta carrera y sin quienes este recorrido hubiera sido bastante más difícil y mucho menos divertido. Santiago, por su parte, le agradece a Nico Rincón, a Paulina, a Alvarito y a Alonso, por estar ahí en los momentos que importaban.

A todos, gracias de verdad.

Resumen

Este documento presenta el diseño, implementación y evaluación de un sistema de detección de intrusiones (IDS) para redes IoT/MQTT desplegado en una arquitectura jerárquica Edge-Fog-Cloud. El sistema combina modelos livianos de *machine learning* ejecutados en microcontroladores ESP32-S3, aprendizaje federado jerárquico (HFL) coordinado desde gateways Raspberry Pi 4 y un servidor PC, y cifrado ligero **ASCON-128** para proteger el ciclo de aprendizaje. La investigación evalúa cuatro dimensiones: viabilidad de TinyML en el borde, reducción de transferencia de datos mediante HFL, overhead criptográfico de ASCON y comparación entre variantes de arquitectura y modelo.

Se diseñó un perceptrón multicapa (MLP) compacto de 1 139 parámetros (≈ 4.5 KB) con arquitectura $13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$, capaz de clasificar flujos de red en tres categorías (**normal**, **mqtt_bruteforce**, **scan_A**) utilizando 13 características extraídas de flujos. El sistema federado comparte únicamente las dos últimas capas del modelo (163 parámetros) y fue evaluado en las ramas **hfl_v7-RN**, **hfl_v7-no-ascon** y **hfl_v7-CNN**, incluyendo modo estándar y modo FOG.

Como línea base, se entrenaron modelos clásicos centralizados, alcanzando hasta 99.94 % de accuracy con Random Forest, aunque con requerimientos de memoria incompatibles con microcontroladores. El notebook general de entrenamiento comparó además MLP/RN, CNN-1D, Residual-MLP y Tiny Transformer; el MLP/RN obtuvo el mejor balance entre precisión offline ($\approx 90.6\%$), tamaño e implementación en ESP32. La cuantificación del overhead de ASCON-128 mostró costo temporal bajo frente a la duración de las rondas federadas y un incremento moderado del tamaño de los mensajes, sin degradación observable de convergencia.

Los resultados validan que la combinación de TinyML, aprendizaje federado y cifrado ligero constituye una solución viable y segura para detección de intrusiones en el borde de redes IoT, ofreciendo un compromiso efectivo entre precisión, privacidad, seguridad y eficiencia computacional.

Keywords: Detección de Intrusiones, IoT, TinyML, Aprendizaje Federado Jerárquico, ASCON-128, Criptografía Ligera, Edge-Fog-Cloud, ESP32-S3, Raspberry Pi, CNN-1D, MLP, Seguridad de Redes.

Tabla de Contenido

List of Figures	x
List of Tables	xi
1. Introducción	1
1.1. Contexto	1
1.2. Problemática	2
1.3. Justificación	3
1.4. Objetivos	4
1.4.1. Objetivo General	4
1.4.2. Objetivos Específicos	4
2. Marco Teórico	5
2.1. Marco Conceptual	5
2.1.1. IDS en redes IoT/IIoT	5
2.1.2. Representación por flujos	5
2.1.3. TinyML y modelos compactos	6
2.1.4. Aprendizaje federado jerárquico	6
2.1.5. Cifrado ligero y ASCON-128	7
2.1.6. Agregación FOG	7
2.2. Estado del Arte	8
2.2.1. IDS y TinyML para IoT/IIoT	8
2.2.2. Aprendizaje federado para seguridad IoT	9
2.2.3. Criptografía ligera y autenticación	9
2.2.4. Datasets y representación del tráfico	10
2.3. Síntesis de la Brecha	10
3. Metodología	12
3.1. Modelo de Amenazas y Suposiciones	12
3.2. Arquitectura del Sistema	13
3.2.1. Capa Edge: Nodos ESP32-S3	13
3.2.2. Capa Fog: Gateway Raspberry Pi 4	14
3.2.3. Capa Cloud: Servidor PC	14
3.2.4. Modos de operación	14

3.3. Conjunto de Datos	15
3.4. Modelo de Aprendizaje: MLP Multiclase	16
3.5. Modelos livianos evaluados	18
3.6. Protocolo de Aprendizaje Federado Jerárquico	18
3.7. Integración de Cifrado Ligero usando ASCON-128	20
3.7.1. Parámetros Criptográficos	21
3.7.2. Canales Cifrados	21
3.7.3. Métricas de Overhead	21
3.8. Modelos Clásicos de ML como Línea Base	22
3.9. Protocolo Experimental	22
3.9.1. Métricas de Evaluación	24
4. Resultados	25
4.1. Desempeño de Modelos Clásicos (Línea Base Centralizada)	25
4.1.1. Viabilidad de Despliegue en ESP32	26
4.2. Comparación de Modelos Livianos	26
4.3. Aprendizaje Federado Jerárquico: Convergencia y Precisión	27
4.3.1. Convergencia del Modelo Federado	27
4.3.2. Evolución temporal de accuracy, loss y pesos	28
4.3.3. Comparación entre variantes hf1	30
4.3.4. Comparación con la literatura	32
4.4. Overhead del Cifrado ASCON-128	33
4.4.1. Overhead Temporal	34
4.4.2. Overhead de Comunicación	35
4.4.3. Resumen: Impacto de ASCON en el Sistema HFL	35
5. Conclusiones y Trabajos Futuros	37
5.1. Conclusiones	37
5.2. Trabajos Futuros	38
Bibliografía	39
A. Anexos	44
A.1. Anexo A: Arquitectura del Sistema HFL	44
A.2. Anexo B: Archivos del Sistema	45
A.3. Anexo C: Repositorio del Proyecto	46
A.4. Anexo D: Resumen de literatura clave incorporada	46

Lista of Figuras

3-1. Modos de operación: (a) estándar con agregación directa PC–gateways y (b) FOG con agregación jerárquica mediante un gateway líder.	15
3-2. Arquitectura del modelo MLP multiclase. Las capas ocultas 3 y de salida (recuadro punteado) conforman las <i>capas federadas</i> (W_3, b_3, W_4, b_4), cuyos 163 parámetros son los únicos intercambiados durante el aprendizaje federado. 17	17
3-3. Diagrama de secuencia del protocolo HFL bidireccional entre Cloud, Fog y Edge. Muestra las fases de despliegue del modelo, entrenamiento local y agregación FedAvg, así como los canales protegidos con ASCON-128 y los topics del modo FOG.	20
4-1. Accuracy y global loss promedio por ronda (1–20) para las 8 variantes implementadas de HFL.	28
4-2. Magnitud promedio de los cuatro pesos federados (<code>w3_mag</code> , <code>w4_normal</code> , <code>w4_brute</code> , <code>w4_scan</code>) por cada variante a lo largo de las rondas 1–20.	29
4-3. Duración promedio por ronda federada para las 8 variantes (rondas 1–20).	31
4-4. Desbalance de <i>accuracy</i> entre gateways ($ acc_A - acc_B $) por ronda y por variante. 31	31
4-5. Latencia p95 por variante y canal. Las variantes PLAIN registran solo el tiempo de serialización JSON; las ASCON incluyen cifrado y descifrado autenticado en cada salto.	33
4-6. Tamaño de payload por variante y canal: plaintext promedio (izquierda) y overhead añadido por el envelope ASCON+Base64 (derecha). Las variantes PLAIN tienen overhead cero.	33
A-1. Arquitectura física del sistema HFL v7 sobre las capas Edge (ESP32-S3), Fog (Raspberry Pi) y Cloud (PC). La línea discontinua indica los enlaces <i>fog/*</i> activos solo en el modo FOG; los detalles por capa y la cobertura de ASCON-128 se describen en el texto.	44

Lista de Tablas

2-1.	Relación entre literatura y arquitectura propuesta del diseño de la red. . . .	11
3-1.	Composición del dataset de flujos de red.	15
3-2.	Vector de 13 features extraídas por flujo.	16
3-3.	Distribución de parámetros del modelo MLP.	17
3-4.	Modelos livianos entrenados y artefactos exportados.	18
3-5.	Canales de comunicación protegidos con ASCON-128.	21
3-6.	Resumen de las variantes experimentales federadas.	23
3-7.	Ejes de evaluación y métricas asociadas.	24
4-1.	Desempeño de modelos clásicos de ML (entrenamiento centralizado).	25
4-2.	Estimación de huella de memoria para despliegue en ESP32.	26
4-3.	Comparación offline de modelos livianos sobre las 13 features.	27
4-4.	Métricas de convergencia del sistema HFL.	27
4-5.	Comparación de accuracy entrenamiento federado vs. centralizado.	29
4-6.	Resumen comparativo de las 8 variantes HFL v7 evaluadas. GW p95 : latencia p95 de descifrado en gateway (ESP32→RPi). Srv dec p95 : latencia p95 de descifrado en servidor (RPi→PC). Overhead : bytes adicionales promedio sobre el plaintext (0 en variantes PLAIN).	30
4-7.	Comparación cualitativa con trabajos relacionados.	32
4-8.	Latencia p95 (ms) promedio de cifrado/descifrado por variante.	34
4-9.	Overhead de tamaño por variante ASCON: plaintext promedio del payload sin cifrar, bytes adicionales del envelope ASCON+Base64, y su ratio sobre el plaintext. Las variantes PLAIN se omiten por tener overhead cero por construcción.	35
4-10.	Resumen del overhead total de ASCON-128 por ronda federada, agregando los promedios de las 4 variantes ASCON y las 4 variantes PLAIN.	35
A-1.	Archivos principales del sistema HFL v7, agrupados por capa de despliegue.	45
A-2.	Literatura clave y aporte concreto a la tesis.	46

1. Introducción

1.1. Contexto

La adopción de redes *Internet of Things* (IoT) e *Industrial IoT* (IIoT) ha aumentado la superficie de ataque en hogares, laboratorios, plantas industriales y sistemas de monitoreo conectados. Estos entornos combinan dispositivos heterogéneos, protocolos livianos como MQTT, conectividad intermitente y capacidades limitadas de cómputo. Estas condiciones dificultan desplegar mecanismos de detección convencionales basados en procesamiento centralizado [1, 2]. El *NIST Cybersecurity Framework* ubica la detección como una función central dentro de la gestión del riesgo. En redes IoT, esta función exige observar el tráfico de forma continua e identificar eventos anómalos antes de que afecten la operación [3].

La literatura reporta buenos resultados en sistemas de detección de intrusiones de red (NIDS) basados en aprendizaje automático cuando los modelos se entrenan y evalúan en condiciones controladas. El problema es que estas evaluaciones suelen asumir condiciones que no siempre existen en IoT: datos centralizados, servidores con alta capacidad de memoria, comunicaciones internas sin protección criptográfica y modelos demasiado grandes para ejecutarse en microcontroladores [4, 5]. Por esta razón, el desafío no es solo maximizar la exactitud, sino también demostrar que la arquitectura puede operar sobre hardware restringido, actualizarse de forma distribuida y proteger los artefactos generados durante el aprendizaje. Trabajos recientes señalan que los enfoques TinyML pocas veces incorporan entrenamiento *on-board* y que, cuando lo hacen, suelen requerir FL o *Transfer Learning* para evitar el envío de datos crudos al servidor [6, 7].

Esta tesis aborda ese reto mediante una arquitectura Edge–Fog–Cloud para detectar intrusiones en tráfico MQTT. La capa Edge se implementa con nodos ESP32-S3 que ejecutan inferencia TinyML y transmiten características de flujo. La capa Fog se implementa sobre una Raspberry Pi 4 como gateway MQTT, entrenador local y agregador intermedio. La capa Cloud se ejecuta en un PC con FastAPI y coordina la agregación global mediante FedAvg. El sistema se evalúa en varias configuraciones: RN/MLP con ASCON-128, RN/MLP sin ASCON como línea base, CNN-1D con ASCON-128, CNN-1D sin ASCON y variantes con agregación Fog entre gateways.

La elección de ASCON-128 se justifica por su adopción reciente como estándar NIST de crip-

tografía ligera para dispositivos restringidos [8, 9]. A diferencia de esquemas *ultra-lightweight* basados solo en permutaciones —los cuales han mostrado vulnerabilidades frente a ataques de divulgación de secretos y desincronización [10]—, ASCON-128 proporciona cifrado autenticado (AEAD) con un overhead predecible. En esta tesis, ASCON-128 se usa para proteger mensajes y pesos del modelo en los canales MQTT y HTTP, garantizando confidencialidad e integridad sin imponer una carga excesiva en los nodos edge.

1.2. Problemática

La primera limitación está en la **viabilidad de despliegue**. Algunos modelos centralizados, como Random Forest o SVM-RBF, pueden obtener precisiones muy altas, pero su tamaño y consumo de memoria dificultan su ejecución en microcontroladores. En IoT, el nodo Edge debe inferir con pocos recursos, bajo consumo y comunicación limitada. La literatura sobre IIoT embebido señala que llevar modelos de IA a plataformas con MCU exige ajustar el modelo, las herramientas y el *stack* de despliegue. También muestra que muchos *frameworks* actuales se concentran en inferencia y no en entrenamiento *on-board* [5, 6]. Por esta razón, un IDS para IoT no debe evaluarse solo por precisión, sino también por memoria, frecuencia de inferencia y costo de comunicación [11, 12].

La segunda limitación está en la **privacidad y distribución de datos**. Enviar capturas PCAP o flujos completos desde varias redes hacia un servidor central aumenta el tráfico y expone información sensible. El aprendizaje federado reduce este riesgo porque comparte pesos o gradientes, no datos crudos. Sin embargo, en IoT aparecen otros problemas: datos *non-IID*, clientes que no completan todas las rondas, diferencias de hardware y sobrecarga por comunicación frecuente [13, 14, 7]. Además, los esquemas FL clásicos suelen asumir agregadores con recursos suficientes. Esto no siempre se cumple cuando parte del procesamiento se acerca al borde de la red. Una arquitectura jerárquica permite separar responsabilidades entre ESP32-S3, Raspberry Pi 4 y PC, y reduce la carga directa sobre los nodos más restringidos [15].

La tercera limitación es la **seguridad del ciclo federado**. Aunque FL evita transferir datos crudos, los pesos del modelo, las métricas locales y los mensajes de control siguen siendo información sensible. En esta tesis, las comunicaciones entre ESP32-S3, Raspberry Pi 4 y PC deben proteger confidencialidad e integridad sin imponer un costo excesivo. ASCON-128 se adapta a las restricciones del sistema porque NIST lo seleccionó como estándar de cifrado ligero para dispositivos con recursos limitados [8, 9]. Proporciona cifrado autenticado con datos asociados (AEAD), ocupa poca RAM y ROM, y mantiene un costo computacional bajo en MCU como el ESP32-S3. Esta elección evita recurrir a esquemas propios no verificados que, basados en operaciones simples o permutaciones débiles, han mostrado vulnerabilidades frente a ataques de divulgación de secretos y desincronización [10].

Por ello, la evaluación del sistema no se reduce a la exactitud del clasificador. Se mide el costo del pipeline completo: rendimiento del modelo TinyML, tráfico del aprendizaje federado jerárquico, sobrecarga de ASCON-128 y el impacto de prescindir de la capa Fog.

1.3. Justificación

La proliferación de dispositivos IoT en el ámbito doméstico e industrial ha ampliado la superficie de exposición a ciberataques. En este escenario, un sistema de detección de intrusiones (IDS) orientado a tráfico MQTT no puede basarse exclusivamente en un modelo centralizado. Los nodos IoT operan con memoria reducida, poca capacidad de cómputo y energía limitada. Además, el envío de flujos de datos sin procesar hacia un servidor remoto no solo incrementa el tráfico de red, sino que también revela información potencialmente sensible sobre la topología local y los patrones de comunicación.

Para enfrentar esta limitación, esta tesis propone un IDS basado en aprendizaje federado jerárquico (HFL). La propuesta retiene los datos dentro del entorno donde se generan y únicamente intercambia parámetros del modelo durante el entrenamiento. Este enfoque disminuye la exposición de información sensible y reduce significativamente el volumen de datos transmitidos, en comparación con una arquitectura centralizada que requeriría el envío de flujos completos o capturas de tráfico.

La validación se realiza sobre componentes físicos reales: un ESP32-S3 actúa como nodo de borde, una Raspberry Pi 4 funciona como *gateway* y capa Fog, y un PC ordinario cumple el rol de coordinador en la nube. El modelo predictivo utiliza 13 características derivadas de cada flujo de red y distingue tres categorías de tráfico: **normal**, **mqtt_bruteforce** y **scan_A**. Esta disposición permite examinar el ciclo completo del sistema: inferencia local en el nodo Edge, entrenamiento periódico en el *gateway*, agregación global mediante el algoritmo FedAvg y posterior redistribución del modelo actualizado.

El cifrado ligero ASCON-128 se integra para resguardar los canales MQTT y HTTP empleados durante las rondas federadas. Su elección responde a su reciente estandarización por parte del NIST como esquema de criptografía ligera, así como a su adecuación a entornos con recursos acotados. Cabe aclarar que ASCON-128 no se introduce aquí como un medio para mejorar la precisión del clasificador, sino como un mecanismo que garantiza confidencialidad e integridad sobre los mensajes, las métricas locales y los pesos del modelo que se transmiten entre las capas.

El plan experimental contempla ocho variantes, que resultan de combinar: presencia o ausencia de ASCON-128, uso o no de la capa de agregación Fog, y dos familias de modelos (RN/MLP frente a CNN-1D). La versión sin ASCON sirve como referencia para aislar el costo criptográfico en términos de latencia y tamaño de mensaje. La variante CNN-1D permite

determinar si una arquitectura con mayor capacidad expresiva mejora el rendimiento respecto al MLP/RN. Por último, el modo Fog incorpora una agregación intermedia en la propia Raspberry Pi 4 antes de alcanzar el servidor central, lo que facilita analizar su impacto sobre la convergencia, los retardos y la sobrecarga de comunicación.

El documento está estructurado de la siguiente manera: el Capítulo 1 presenta el problema y los objetivos; el Capítulo 2 resume el marco conceptual y el estado del arte; el Capítulo 3 describe la arquitectura y el protocolo experimental; el Capítulo 4 presenta resultados; el Capítulo 5 concluye y plantea trabajos futuros; y los anexos resumen archivos y artefactos del sistema. Adicionalmente, si desea consultar los scripts de las arquitecturas implementadas en Python y C++, puede acceder a ellos a través del siguiente enlace: **Repositorio del proyecto**.

1.4. Objetivos

1.4.1. Objetivo General

Diseñar, implementar y evaluar un prototipo funcional de un sistema de detección de intrusiones en el borde que combine modelos TinyML para inferencia eficiente, aprendizaje federado para entrenamiento colaborativo y cifrado ligero para proteger el intercambio de parámetros y mensajes.

1.4.2. Objetivos Específicos

1. Construir el pipeline de datos para la captura, normalización y generación de features a partir del tráfico y/o de flujos de red, y seleccionar un conjunto de datasets públicos adecuados para entrenamiento y evaluación comparativa.
2. Entrenar y comparar modelos compactos para IDS IoT, incluyendo MLP, CNN-1D, Residual-MLP y Tiny Transformer, exportando los pesos y los parámetros de normalización para ejecución en el microcontrolador ESP32.
3. Implementar un esquema de aprendizaje federado para el entrenamiento distribuido de modelos, evaluando estrategias de agregación, la estabilidad con datos no IID y los costos de comunicación, apoyándose en criterios para la selección de frameworks y extensiones.
4. Integrar mecanismos de cifrado ligero y de autenticación en la arquitectura propuesta y validar experimentalmente el sistema completo, evaluando la capacidad de detección de ataques en los nodos, el impacto del aprendizaje federado frente al entrenamiento centralizado y los efectos en la latencia, el consumo y el desempeño general.

2. Marco Teórico

2.1. Marco Conceptual

2.1.1. IDS en redes IoT/IIoT

Un sistema de detección de intrusiones de red (NIDS) analiza el tráfico y los eventos para identificar actividades maliciosas o anómalas. En redes IoT/IIoT, esta tarea presenta restricciones adicionales frente a una red tradicional. Los dispositivos suelen tener poca memoria, bajo poder de cómputo, consumo energético limitado y comunicación intermitente. Además, usan protocolos livianos y generan tráfico heterogéneo según el tipo de sensor, actuador o aplicación.

Por esta razón, un IDS para IoT no debe evaluarse solo por el *accuracy*. También debe analizarse su latencia, el uso de memoria, el costo de comunicación y la estabilidad durante la operación [11, 12]. La literatura reciente sobre seguridad IIoT con aprendizaje automático muestra que las capas del sensor, el gateway, el fog y el cloud cumplen funciones distintas. Esto dificulta una solución monolítica satisfactoria y favorece arquitecturas jerárquicas con responsabilidades distribuidas en la red [2, 1]. Otros trabajos han usado SDN, NFV y los modelos de aprendizaje automático para monitoreo y respuesta ante ataques [4, 16], así como enfoques basados en el cloud para gestionar la seguridad IoT de los dispositivos [17]. Sin embargo, estos enfoques no siempre validan el despliegue en microcontroladores reales.

2.1.2. Representación por flujos

El tráfico de red puede representarse mediante paquetes individuales, capturas PCAP, registros de eventos o flujos. Este proyecto de grado utiliza flujos unidireccionales ya que ofrecen una representación compacta y compatible con el procesamiento en el borde. Cada flujo se resume en 13 características numéricas asociadas con el número de paquetes, el tiempos entre paquetes, el tamaños de paquete, los bytes totales del paquete y las banderas TCP.

Esta representación sacrifica parte del detalle temporal de una captura PCAP completa, pero es una decisión de diseño y no un descuido. El objetivo es obtener variables que puedan calcularse y transmitirse desde un ESP32-S3 sin exceder sus recursos. Así, el sistema reduce el volumen de comunicación y mantiene una entrada adecuada para modelos TinyML.

2.1.3. TinyML y modelos compactos

TinyML agrupa técnicas para ejecutar modelos de aprendizaje automático en hardware restringido. En el contexto de esta tesis, el criterio de diseño es que el modelo pueda ejecutarse directamente en microcontroladores ESP32-S3 sin depender de aceleradores externos ni transferir los datos crudos al servidor. La revisión de Dini *et al.* [5] resume el ecosistema actual de *toolchains* (TensorFlow Lite Micro, STM32Cube.AI, Vitis AI, etc.) y muestra que la integración exitosa depende tanto de la arquitectura del modelo como del soporte HW/SW del MCU. Ficco *et al.* [6] reportan un esquema de FL con *transfer learning* y entrenamiento *on-board* en dispositivos como ESP32 y Arduino MKR1010. En su evaluación, la combinación FL+TL alcanzó 86.48% de accuracy en clasificación, con mediciones de latencia y consumo dentro del rango operativo de los dispositivos. Este resultado respalda el uso de entrenamiento local parcial en TinyML, aunque cada arquitectura debe validarse en el hardware y protocolo específicos del despliegue. Los modelos evaluados en esta tesis incluyen MLP, CNN-1D, Residual-MLP y Tiny Transformer; sin embargo, el despliegue principal utiliza un MLP compacto y una CNN-1D debido a su equilibrio entre tamaño, precisión y simplicidad de inferencia, en línea con las recomendaciones de la literatura para clasificación tabular en MCUs [18, 5].

2.1.4. Aprendizaje federado jerárquico

El aprendizaje federado (FL) entrena modelos de forma distribuida sin enviar datos crudos al servidor central. En lugar de transferir capturas o flujos completos, cada cliente comparte actualizaciones del modelo. Este enfoque debe adaptarse en IoT porque los nodos tienen recursos limitados y la comunicación no siempre es estable. Por esta razón, una arquitectura jerárquica (HFL) resulta adecuada, en la que los nodos Edge se comunican con gateways cercanos, y los gateways envían actualizaciones a un coordinador global. [19, 13].

En esta tesis, el ESP32-S3 ejecuta inferencia TinyML y transmite características de flujo. La Raspberry Pi 4 actúa como gateway/Fog, recibe esas características, realiza entrenamiento local y envía actualizaciones al servidor. El PC coordina la agregación global mediante FedAvg. Esta distribución evita cargar el entrenamiento completo sobre el microcontrolador y permite separar las tareas de inferencia, entrenamiento local y agregación global.

La literatura muestra que FL puede aplicarse a IDS en IoT, pero también señala límites importantes. Hajj *et al.* [20] reportan un IDS federado *cross-layer* con menor consumo de comunicación frente a enfoques centralizados. Chaurasia *et al.* [21] muestran beneficios al usar redes residuales en clientes FL para IIoT. Estos trabajos respaldan el uso de entrenamiento distribuido, aunque ninguno resuelve por completo el despliegue sobre microcontroladores reales.

En entornos IoT reales, Federal Learning lidia con datos *non-IID* entre nodos, desconexiones intermitentes, hardware heterogéneo y overhead de comunicación que se acumula ronda a ronda [14, 7]. En esta tesis la condición *non-IID* se representa de forma parcial: un nodo genera tráfico normal y otro genera una mezcla de tráfico normal, fuerza bruta MQTT y escaneo. No se implementa tolerancia bizantina ni consenso distribuido. Tran *et al.* [15] reportan que varios métodos bizantino-resilientes requieren entre 512 MB y 2 GB de memoria, mientras que MCUs comerciales típicos disponen de alrededor de 264 KB de SRAM. Esa brecha es lo que justifica separar el sistema por capas.

2.1.5. Cifrado ligero y ASCON-128

Reducir el envío de datos crudos no elimina los riesgos de seguridad. En cada ronda se transmiten pesos del modelo, métricas locales y mensajes de control; si esa información es interceptada o modificada, el entrenamiento puede degradarse y el proceso local queda expuesto. Por eso esta tesis integra ASCON-128 como cifrado autenticado ligero para proteger confidencialidad e integridad en los canales MQTT y HTTP del ciclo federado.

ASCON fue seleccionado por NIST en febrero de 2023 como base para el estándar de criptografía ligera, después de un proceso de evaluación con candidatos como GIFT-COFB, Sparkle, Romulus y TinyJambu [8, 9]. La familia ASCON usa una permutación interna de 320 bits organizada en cinco palabras de 64 bits. Entre sus variantes se encuentran ASCON-128, con *rate* de 64 bits, y ASCON-128a, con *rate* de 128 bits. El trabajo de Kaur *et al.* [9] resume implementaciones en dispositivos como FPGA y ASIC, ataques diferenciales, ataques de canal lateral y posibles contramedidas. Esto lo convierte en una referencia útil para sistemas embebidos que procesan mensajes pequeños, como características del flujo de la red, los pesos parciales y los modelos globales desplegados [8, 22].

Una alternativa de bajo costo son los protocolos *ultra-lightweight* basados en operaciones simples, como XOR, rotaciones y desplazamientos. El problema es que varios de estos diseños dependen de permutaciones *ad hoc* y no ofrecen las mismas garantías que un esquema AEAD estandarizado. Mirzaali *et al.* [10] muestran, en una evaluación con tres Raspberry Pi, posibles vulnerabilidades asociadas con divulgación de secretos y desincronización de los dispositivos. En esta tesis, ASCON-128 se compara contra una versión funcionalmente equivalente sin cifrado, con el fin de medir su costo en latencia y el tamaño del mensaje dentro de la arquitectura Edge-Fog-Cloud implementada [23].

2.1.6. Agregación FOG

El modo FOG añade una capa de agregación entre los gateways Raspberry Pi 4 antes de enviar actualizaciones al servidor central. En esta configuración, un gateway líder recibe por

MQTT las actualizaciones de otros gateways, aplica un FedAvg intermedio y transmite al PC una actualización preagregada. Así, el servidor no recibe cada actualización de forma aislada, sino un resultado consolidado en la capa Fog.

Este esquema aparece en contextos con varias zonas o segmentos de red donde los gateways coordinan parte del entrenamiento antes de llegar a la nube. La preagregación cerca de la fuente reduce la dependencia del servidor central y organiza el tráfico de los nodos [24], y hay propuestas que distribuyen la carga según la topología cuando los dispositivos tienen recursos limitados [15].

En esta tesis la jerarquía tiene tres niveles: ESP32-S3 para inferencia y envío de características, Raspberry Pi 4 para entrenamiento local y agregación FOG, y PC para coordinación global. El modo FOG no usa blockchain ni consenso distribuido. Lo que se quiere medir es si meter una agregación intermedia cambia la convergencia, la latencia y el costo de comunicación respecto al modo NoFOG.

2.2. Estado del Arte

La revisión sigue la estructura del sistema propuesto: detección en el borde, aprendizaje federado con capa Fog, protección criptográfica y selección de datos. Organizarla así permite leer el estado del arte desde la arquitectura completa y no solo desde el clasificador. Se parte de trabajos sobre IDS y TinyML en redes IoT/IIoT, se pasa por enfoques federados y jerárquicos para entrenamiento distribuido, y se cierra con mecanismos de cifrado ligero y criterios de selección de datos para validar el sistema.

2.2.1. IDS y TinyML para IoT/IIoT

Los IDS para IoT/IIoT no pueden evaluarse solo por métricas como la precisión. En estos entornos, un modelo útil debe ajustarse a la memoria disponible del dispositivo, responder con baja latencia y mantener un consumo energético compatible con los nodos embebidos. Por eso, trabajos recientes sobre TinyML aplicado a IDS recomiendan reportar métricas de clasificación junto con la memoria, los tiempo de inferencia y la energía consumida [11, 25, 12, 18].

Las revisiones sobre seguridad IoT con aprendizaje profundo, big data e IIoT muestran una limitación recurrente: muchos pipelines se entrenan y se evalúan en servidores con suficientes recursos, pero no llegan a validar el despliegue sobre microcontroladores reales [1, 2]. Esta diferencia es relevante para esta tesis, porque el nodo Edge usa un microcontrolador ESP32-S3 y no un servidor embebido de mayor capacidad.

Dini *et al.* [5] revisan herramientas usadas para llevar modelos a plataformas embebidas, entre

ellas TFLM, STM32Cube.AI y Vitis AI. Su análisis muestra que el despliegue de TinyML exige ajustar el modelo, la conversión y el entorno de ejecución. En la práctica, el problema no termina al obtener una buena métrica de clasificación; también hay que verificar si el modelo cabe, se puede ejecutar y se comunica dentro de las restricciones del dispositivo evaluado.

2.2.2. Aprendizaje federado para seguridad IoT

El aprendizaje federado se ha usado en seguridad IoT porque permite entrenar modelos sin enviar datos crudos a un servidor central. En un IDS, esta ventaja es relevante porque los flujos de red pueden revelar dispositivos, servicios y patrones de operación. Sin embargo, FL también introduce dificultades: datos *non-IID*, clientes con participación parcial, enlaces inestables y sobrecarga por el intercambio de actualizaciones [13, 14]. Los trabajos recientes muestran este compromiso: el entrenamiento federado puede perder precisión frente a un esquema centralizado, pero reduce la transferencia de datos y mejora la privacidad del proceso de entrenamiento [20, 26].

Hajj *et al.* [20] proponen un IDS federado *cross-layer* para IoT, mientras que Chaurasia *et al.* [21] evalúan redes residuales como clientes FL en IIoT. Ficco *et al.* [6] validan FL combinado con *Transfer Learning* para entrenamiento *on-board* en MCUs y Raspberry Pi 3, y lo comparan con TFLite. EdgeTrust-Shard [15] muestra que defensas bizantino-resilientes como Krum, FLTrust o Fed-NGA pueden requerir más memoria de la disponible en participantes basados en MCU. Esto respalda una arquitectura jerárquica donde el ESP32-S3, la Raspberry Pi 4 y el PC asumen tareas distintas dentro del ciclo federado.

2.2.3. Criptografía ligera y autenticación

La criptografía ligera busca proteger comunicaciones en dispositivos con memoria, cómputo y energía limitados. En un IDS federado, esta protección es necesaria porque las actualizaciones del modelo, métricas locales y mensajes de control también son activos sensibles. ASCON-128 es relevante para este escenario porque fue seleccionado por NIST como estándar de criptografía ligera y permite proteger confidencialidad e integridad sin depender de esquemas propietarios o *ad hoc* [8, 22, 9].

Kaur *et al.* [9] presentan una revisión de implementaciones FPGA/ASIC, ataques diferenciales, ataques *cube*, inyección de fallas, ataques de canal lateral y contramedidas para ASCON. En contraste, Mirzaali *et al.* [10] muestran que algunos protocolos ultra-ligeros basados en permutaciones *ad hoc* para RFID/IoT pueden romperse experimentalmente en hardware con Raspberry Pi y Wi-Fi. Otros trabajos proponen control de acceso [27] o cifrado por atributos [23] para dominios como salud y ciudades inteligentes. En los trabajos revisados, estos mecanismos no se integran con un ciclo FL completo sobre ESP32-S3, Raspberry Pi 4 y

PC.

2.2.4. Datasets y representación del tráfico

Los repositorios públicos de tráfico IoT etiquetado y mas usados como: UNSW-NB15, Bot-IoT, TON-IoT, Edge-IIoTset, IoT-23 y CICIoT2023, logran cubrir diversos protocolos y ataques [28, 29, 30, 31, 32, 33, 34]. El riesgo de sobreajuste aparece cuando el modelo se entrena con una única fuente. Para mitigarlo, varios trabajos recientes recomiendan combinar conjuntos o evaluar su compatibilidad antes del entrenamiento [35, 36].

En esta tesis no se utilizan los datasets completos, sino un subconjunto seleccionado que contiene flujos unidireccionales de tres clases: `normal`, `mqtt_bruteforce` y `scan_A`. La decisión no busca representar todo el espectro de ataques IoT, sino reproducir un escenario MQTT concreto, del tipo que puede observarse en un laboratorio de pruebas o en una red doméstica inteligente. Cada flujo se reduce a 13 características numéricas (conteo de paquetes, tiempos entre paquetes, tamaños y banderas TCP) que pueden calcularse directamente en el ESP32-S3 sin saturar su memoria. Esta representación tabular evita transmitir capturas PCAP completas y mantiene la entrada dentro de los límites de inferencia de los modelos TinyML evaluados.

2.3. Síntesis de la Brecha

La literatura revisada aborda por separado TinyML, aprendizaje federado y criptografía ligera. La brecha que esta tesis busca cerrar es la integración y medición conjunta de esas tres dimensiones dentro de un ciclo Edge-Fog-Cloud ejecutado sobre hardware real. El sistema integra inferencia en ESP32-S3, entrenamiento local en Raspberry Pi 4, agregación global en un PC que actúa como coordinador, protección de canales con ASCON-128, una línea base sin cifrado (PLAIN), una variante de modelo CNN-1D y un modo de agregación intermedia entre gateways (FOG). Así, la evaluación se concentra en la arquitectura completa y no solo en la exactitud del clasificador entrenado.

Un aporte técnico del trabajo es la implementación compatible de ASCON-128 en C++ para el nodo Edge y en Python para los niveles Fog y Cloud. La instrumentación registra, por canal y por ronda federada, la latencia de las operaciones criptográficas y el tamaño de los mensajes protegidos. Esto permite aislar el costo de ASCON-128 frente a una versión PLAIN funcionalmente equivalente, ejecutada bajo las mismas condiciones de hardware, software y red.

El diseño experimental compara además dos familias de modelos federables, RN/MLP y CNN-1D, ambas sometidas al mismo protocolo HFL. El modo FOG incorpora una agre-

gación intermedia entre gateways Raspberry Pi 4 con roles de líder y par. En los trabajos revisados, que esta en la Tabla **2-1**, no se encontró una evaluación que combine esta jerarquía de tres niveles, cifrado autenticado ligero en cada salto, una línea base sin protección y el despliegue sobre un microcontrolador real junto con una Raspberry Pi y un PC. Esta combinación define el espacio de evaluación y constituye la contribución experimental del trabajo.

Table 2-1.. Relación entre literatura y arquitectura propuesta del diseño de la red.

Línea	Referencias	Uso en la tesis
TinyML / IDS IoT	[11, 12, 25, 18, 5]	Selección de modelos compactos, reporte de huella de memoria, latencia de inferencia en ESP32-S3 y elección de <i>toolchain</i> de despliegue.
Surveys IoT/IIoT seguridad	[1, 2, 4, 16]	Encuadre del problema, justificación de jerarquía Edge-Fog-Cloud y comparación contra enfoques no edge.
FL / HFL para IDS	[19, 13, 14, 7, 20, 26, 21, 6]	Diseño de FedAvg jerárquico, discusión de datos <i>non-IID</i> y sustento del entrenamiento on-board en MCUs/RPi.
Robustez/escalabilidad[15] FL en MCUs		Justificación de la jerarquía y delimitación de las amenazas no cubiertas (clientes bizantinos).
Criptografía ligera	[8, 22, 9, 10, 37]	Integración de ASCON-128, comparación contra baseline sin cifrado, justificación de no usar permutaciones <i>ad-hoc</i> y delimitación de canales laterales como trabajo futuro.
Datasets IoT-IDS	[28, 34, 32, 33, 35, 36]	Selección de las 13 features de flujo y de las tres clases (<code>normal</code> , <code>mqtt_bruteforce</code> , <code>scan_A</code>).
Pre-agregación / data fusion	[24]	Justificación conceptual del modo FOG (FedAvg de gateways antes del PC).

3. Metodología

La presente investigación sigue un enfoque experimental y cuantitativo, estructurado en seis fases: (i) construcción del pipeline de datos, (ii) entrenamiento y comparación offline de modelos livianos, (iii) implementación de la arquitectura jerárquica de aprendizaje federado, (iv) integración de cifrado ligero ASCON-128, (v) construcción de una línea base sin cifrado y (vi) evaluación de variantes con MLP/RN, CNN-1D y estrategia FOG sobre hardware real.

3.1. Modelo de Amenazas y Suposiciones

Antes de detallar la arquitectura, se hace explícito el modelo de amenazas que motiva las decisiones de diseño. Se asume un atacante de red pasivo o activo capaz de observar y modificar el tráfico inalámbrico entre nodos ESP32, gateways Raspberry Pi y el PC coordinador, pero *externo* al sistema. Las amenazas que el sistema busca mitigar son:

- **Confidencialidad de pesos y features:** Un atacante que capture los mensajes MQTT/HTTP no debe poder leer el contenido (pesos federados, métricas locales, vectores de features). Esto se garantiza con ASCON-128 sobre todos los canales [8, 9].
- **Integridad y autenticidad:** Mensajes alterados o inyectados deben ser detectados y descartados. Se usa el tag de autenticación de 128 bits que ASCON-128 produce de forma nativa, evitando el patrón frecuente de combinar cifrado simétrico con HMAC externo [37].
- **Reuso de mensajes (replay):** Cada nonce se construye a partir de un timestamp truncado a 32 bits y un contador monótono local; combinado con el tag autenticado, esto detecta reutilizaciones aunque un atacante reenvíe un mensaje válido.
- **Diseños criptográficos no estandarizados:** Se descarta el uso de permutaciones ultra-ligeras *ad-hoc* para autenticación, dado que análisis recientes han demostrado vulnerabilidades estructurales explotables sobre Raspberry Pi en condiciones realistas [10].

Amenazas fuera del alcance. Esta versión del sistema no mitiga las siguientes amenazas:

- **Clientes bizantinos y *data poisoning*:** Un gateway comprometido podría enviar pesos manipulados al servidor. Un nodo Edge comprometido, aunque no entrene el modelo, podría enviar características falsas al gateway y contaminar el conjunto local. Tran *et al.* [15] reportan que un 10 % de participantes maliciosos puede degradar la *accuracy* global de 96 % a 23 %. Defensas como Krum, FLTrust, Fed-NGA o MF-PoP

requieren memoria y cómputo que no siempre son viables en participantes basados en MCU. Esta protección queda como trabajo futuro.

- **Ataques de canal lateral sobre ASCON:** Ataques por consumo de potencia, emisiones electromagnéticas o inyección de fallas pueden recuperar claves si el atacante tiene acceso físico al dispositivo [9]. Esta tesis asume un entorno físico controlado y no evalúa resistencia frente a canal lateral.
- **Inferencia de membresía:** FL evita compartir datos crudos, pero los pesos del modelo pueden filtrar información sobre los datos locales. En esta versión no se aplican defensas como privacidad diferencial o DP-SGD.
- **Compromiso del PC coordinador:** El PC se considera parte confiable del sistema. Si el coordinador se compromete, puede alterar la agregación global o redistribuir modelos maliciosos. Enfoques con blockchain o múltiples coordinadores podrían reducir este supuesto, pero no se implementan en esta tesis [15].

3.2. Arquitectura del Sistema

El sistema se organiza en tres capas: Edge, Fog y Cloud. Esta división separa las tareas según la capacidad de cada dispositivo. El ESP32-S3 realiza inferencia y envío de características; la Raspberry Pi 4 recibe los flujos, entrena localmente y puede agregar actualizaciones; el PC coordina la agregación global y redistribuye el modelo actualizado.

3.2.1. Capa Edge: Nodos ESP32-S3

La capa Edge usa microcontroladores **ESP32-S3** con conectividad Wi-Fi. Estos nodos representan el punto más restringido del sistema y ejecutan las tareas que deben ocurrir cerca del tráfico MQTT. Cada nodo cumple tres funciones:

1. **Generación de flujos MQTT:** produce flujos con patrones estadísticos calibrados a partir de los datasets usados en la Sección 3.3.
2. **Extracción de características:** calcula un vector de 13 características por flujo, descritas en la Tabla 3-2.
3. **Inferencia local TinyML:** ejecuta un modelo compacto en el microcontrolador y clasifica cada flujo como `normal`, `mqtt_bruteforce` o `scan_A`.

Se usan dos tipos de nodo. El nodo *normal* genera solo tráfico benigno. El nodo *simulado* genera una mezcla de 40 % tráfico normal, 30 % `mqtt_bruteforce` y 30 % `scan_A`. Esta separación permite representar una distribución no uniforme entre nodos, útil para evaluar el comportamiento del ciclo federado. En la rama RN se usa un modelo MLP. En la rama CNN se usa una CNN-1D con capas convolucionales fijas y capas densas federadas.

3.2.2. Capa Fog: Gateway Raspberry Pi 4

La Raspberry Pi 4 actúa como *gateway* Fog y ejecuta `gateway_hfl.py` o `gateway_hfl_fog.py`, según el escenario experimental. Sus responsabilidades incluyen:

- Actuar como **broker MQTT** (Mosquitto) para recibir las features de los ESP32.
- Mantener un **buffer de entrenamiento** de $N_s = 30$ muestras etiquetadas mediante una función heurística basada en las distribuciones reales del dataset.
- **Entrenar localmente** un modelo Keras/TensorFlow idéntico al del ESP32 (5 épocas, Adam con $\eta = 0,005$, batch=8).
- **Transmitir los pesos** de las capas federadas al servidor central vía HTTP.
- **Distribuir el modelo global** recibido del servidor a los ESP32 vía MQTT.
- En modo FOG, **agregar pesos entre gateways Raspberry Pi** antes de enviar una actualización preagregada al PC.

3.2.3. Capa Cloud: Servidor PC

Un PC convencional ejecuta `server_hfl.py` o `server_hfl_fog.py` (FastAPI + Uvicorn) y coordina el aprendizaje federado global:

- Recibe los pesos locales de K gateways (en esta implementación, $K = 2$).
- Aplica el algoritmo **FedAvg** (Sección 3.6) ponderado por número de muestras.
- Distribuye el modelo global actualizado a todos los gateways.
- Proporciona un **dashboard analítico** en tiempo real con gráficas de accuracy y loss por ronda.

3.2.4. Modos de operación

La operación de la arquitectura fue implementada en dos modos:

- **Modo estándar:** cada Raspberry Pi actúa como gateway independiente. El servidor PC espera actualizaciones de $K = 2$ gateways y aplica FedAvg global.
- **Modo FOG:** una Raspberry Pi líder recibe pesos de una Raspberry Pi par mediante MQTT, calcula FedAvg local de la capa fog y envía al PC una actualización preagregada. El modelo global retorna al líder y se redistribuye a los peers y a los ESP32.

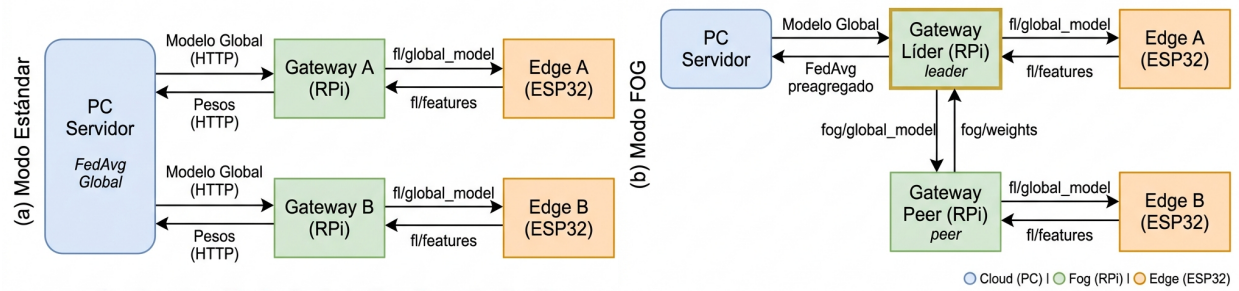


Figure 3-1.. Modos de operación: (a) estándar con agregación directa PC–gateways y (b) FOG con agregación jerárquica mediante un gateway líder.

3.3. Conjunto de Datos

Se utilizan tres datasets públicos de flujos de red IoT, cada uno representando una clase de tráfico. Estos provienen de capturas PCAP procesadas y convertidas a representaciones de flujo unidireccional (*uniflow*):

Table 3-1.. Composición del dataset de flujos de red.

Dataset	Flujos	Etiqueta	Protocolo principal
uniflow_normal.csv	171 837	0 (normal)	MQTT, TCP
uniflow_mqtt_bruteforce.csv	33 080	1 (mqtt_bruteforce)	MQTT (login)
uniflow_scan_A.csv	51 359	2 (scan_A)	TCP SYN
Total	256 276	3 clases	—

Cada flujo se representa mediante un vector de $d = 13$ características numéricas, seleccionadas para ser computables en tiempo real por un microcontrolador:

Table 3-2.. Vector de 13 features extraídas por flujo.

Índice	Feature	Descripción
0	num_pkts	Número de paquetes del flujo
1	mean_iat	Inter-arrival time promedio (s)
2	std_iat	Desviación estándar del IAT
3	min_iat	IAT mínimo
4	max_iat	IAT máximo
5	mean_pkt_len	Longitud promedio de paquete (bytes)
6	num_bytes	Total de bytes del flujo
7	num_psh_flags	Conteo de flags PSH
8	num_rst_flags	Conteo de flags RST
9	num_urg_flags	Conteo de flags URG
10	std_pkt_len	Desviación estándar de longitud de paquete
11	min_pkt_len	Longitud mínima de paquete
12	max_pkt_len	Longitud máxima de paquete

Para el preprocesamiento, se aplica un **StandardScaler** que normaliza cada feature a media cero y varianza unitaria. Los parámetros del scaler (μ_i, σ_i) se exportan como constantes embebidas en el firmware del ESP32, permitiendo la normalización en el dispositivo sin dependencia de bibliotecas externas.

3.4. Modelo de Aprendizaje: MLP Multiclase

Se diseñó un perceptrón multicapa (MLP) compacto orientado a la ejecución en microcontroladores con restricciones de memoria:

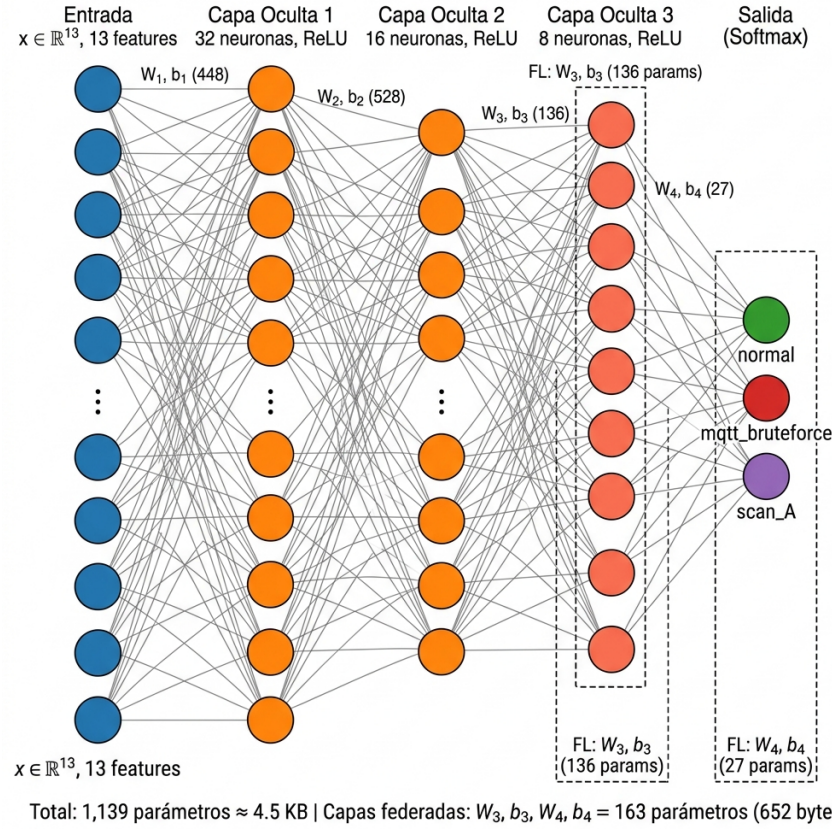


Figure 3-2.. Arquitectura del modelo MLP multiclase. Las capas ocultas 3 y de salida (recuadro punteado) conforman las *capas federadas* (W_3, b_3, W_4, b_4), cuyos 163 parámetros son los únicos intercambiados durante el aprendizaje federado.

Las primeras tres capas utilizan activación ReLU y la capa de salida emplea Softmax para obtener probabilidades sobre las 3 clases. La Tabla **3-3** detalla la distribución de parámetros:

Table 3-3.. Distribución de parámetros del modelo MLP.

Capa	Entrada	Salida	Parámetros
Dense 1 (ReLU)	13	32	$13 \times 32 + 32 = 448$
Dense 2 (ReLU)	32	16	$32 \times 16 + 16 = 528$
Dense 3 (ReLU)	16	8	$16 \times 8 + 8 = 136$
Dense 4 (Softmax)	8	3	$8 \times 3 + 3 = 27$
Total			1 139

Con 1 139 parámetros en punto flotante de 32 bits, el modelo completo ocupa ≈ 4.5 KB de memoria, lo que permite su ejecución directa en la SRAM del ESP32-S3 sin necesidad de cuantización.

En el esquema de aprendizaje federado, solo las **dos últimas capas** (W_3, b_3, W_4, b_4 , 163

parámetros) son intercambiadas entre nodos, reduciendo el costo de comunicación a ≈ 652 bytes por actualización (sin serialización JSON).

3.5. Modelos livianos evaluados

Antes del despliegue federado se ejecutó un experimento base de entrenamiento con cuatro familias de modelos sobre las mismas 13 características y las mismas tres clases. El objetivo fue seleccionar una arquitectura viable para ESP32-S3 y, al mismo tiempo, documentar alternativas para comparación experimental:

Table 3-4.. Modelos livianos entrenados y artefactos exportados.

Modelo	Arquitectura	Artefactos exportados
MLP/RN	$13 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 3$	Pesos, modelo Keras, scaler
CNN-1D	Conv1D(32) $\rightarrow$ Conv1D(16) $\rightarrow$ GAP $\rightarrow$ Dense8 $\rightarrow$ Dense3	Pesos Conv fusionados + capas densas federadas
Residual-MLP	MLP compacto con conexión residual	Pesos, scaler y modelo Keras (offline)
Tiny Transformer	Auto-atención tabular liviana	Pesos y modelo Keras (offline)

El despliegue principal usa el MLP/RN, ya que combina una arquitectura pequeña con una implementación directa en C++ sobre el ESP32-S3. Su entrada corresponde a las 13 características de flujo y su salida cubre las tres clases del IDS. Además, sus capas densas permiten serializar los pesos y agregarlos mediante FedAvg sin incorporar operadores adicionales en el nodo edge.

La CNN-1D se mantiene como variante experimental. En esta rama, las capas convolucionales permanecen fijas en el edge y solo se federan las capas densas `dense_1` y `dense_out`. Esta decisión reduce el número de parámetros enviados en cada ronda y separa la extracción local de patrones de la etapa de clasificación. La actualización parcial de capas finales se ha usado en trabajos de FL+TL para TinyML [6, 26]. En esta tesis, se aplica para limitar el tamaño del payload y el costo de cómputo durante el entrenamiento federado.

3.6. Protocolo de Aprendizaje Federado Jerárquico

El sistema implementa aprendizaje federado jerárquico (HFL) sobre la arquitectura Edge-Fog-Cloud. El flujo separa tres tareas: captura de características en el ESP32-S3, entrenamiento local en el gateway Raspberry Pi 4 y agregación global en el coordinador cloud.

1. **Nivel Edge (ESP32 $\rightarrow$ RPi):** Los nodos ESP32-S3 publican cada 5 segundos los vectores de 13 características de flujo mediante MQTT en el tópico `f1/features`. El gateway recibe estos mensajes y acumula un buffer de $N_s = 30$ muestras etiquetadas heurísticamente.
2. **Nivel Fog (RPi $\rightarrow$ PC):** Cuando el buffer alcanza N_s muestras, la Raspberry Pi 4 ejecuta entrenamiento local durante $E = 5$ épocas. Al finalizar, envía al coordinador las capas federadas, la accuracy local y el número de muestras usadas en la ronda. En una configuración jerárquica alternativa, este envío puede dirigirse primero a una RPi líder.
3. **Nivel Cloud (Agregación FedAvg):** El coordinador espera las actualizaciones de $K = 2$ gateways y aplica FedAvg ponderado por el número de muestras:

$$W_{\text{global}}^{(t+1)} = \frac{\sum_{k=1}^K n_k \cdot W_k^{(t)}}{\sum_{k=1}^K n_k} \quad (3-1)$$

donde n_k es el número de muestras usadas para el entrenamiento local del gateway k y $W_k^{(t)}$ sus pesos locales en la ronda t .

El modelo global resultante es distribuido de vuelta a los gateways (HTTP POST a `/deploy-model`), quienes a su vez lo publican a los ESP32 por MQTT (`f1/global_model`). La Figura **3-3** resume el flujo bidireccional completo en sus tres fases canónicas (top-down deployment, recolección y entrenamiento local, y agregación bottom-up).

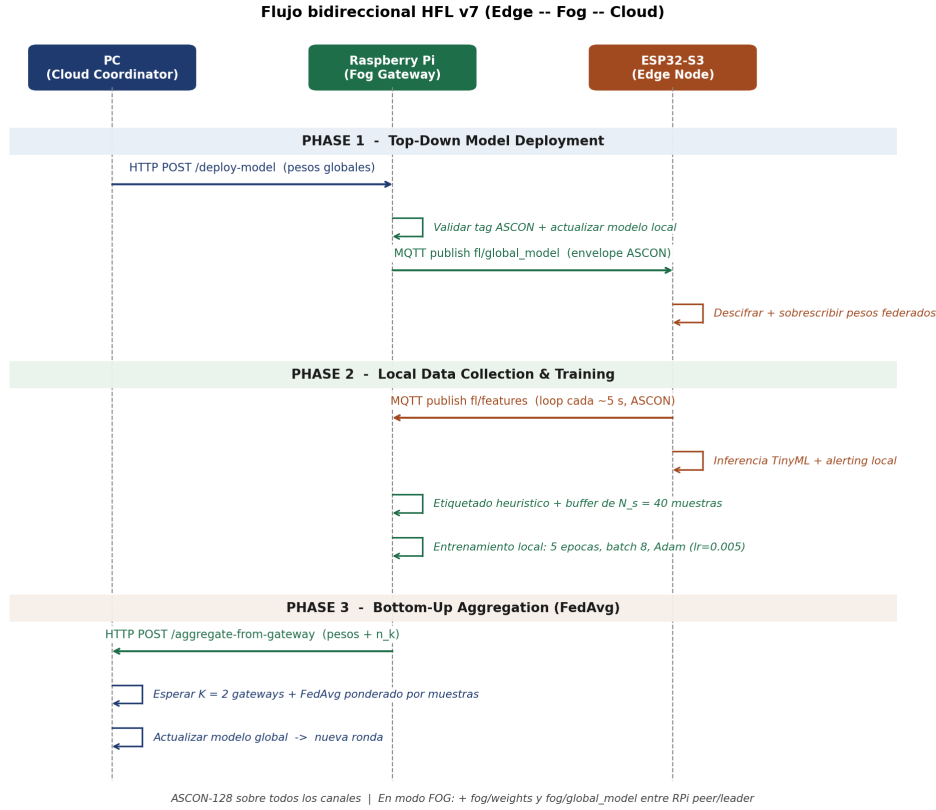


Figure 3-3.. Diagrama de secuencia del protocolo HFL bidireccional entre Cloud, Fog y Edge. Muestra las fases de despliegue del modelo, entrenamiento local y agregación FedAvg, así como los canales protegidos con ASCON-128 y los topics del modo FOG.

En modo FOG, el flujo agrega dos canales adicionales: **fog/weights** para que los peers envíen pesos al líder y **fog/global_model** para que el líder redistribuya el modelo global a los peers. El etiquetado en el gateway se realiza mediante una **función heurística** calibrada con las distribuciones estadísticas reales de los datasets:

- Si $\text{num_pkts} \geq 50$ y $\text{num_psh} \geq 10$: etiqueta = 1 (bruteforce).
- Si $\text{num_pkts} \leq 5$ y $\text{mean_pkt_len} \leq 50$ y $\text{num_psh} \leq 1$: etiqueta = 2 (scan_A).
- Si $\text{num_pkts} \leq 30$ y $\text{mean_pkt_len} \geq 50$: etiqueta = 0 (normal).
- En otro caso, la muestra se descarta (-1).

3.7. Integración de Cifrado Ligero usando ASCON-128

Esta tesis usa **ASCON-128** para proteger los mensajes que circulan entre el edge, el fog y el cloud. El cifrado cubre los artefactos enviados por MQTT y HTTP: vectores de características, actualizaciones federadas y modelos globales. Al operar como esquema AEAD, cada mensaje cifrado incluye verificación de integridad y autenticidad además de confidencialidad [8].

3.7.1. Parámetros Criptográficos

- **Clave simétrica:** 128 bits (16 bytes), pre-compartida entre los dispositivos del prototipo.
- **Nonce:** 128 bits, generado a partir de un timestamp truncado a 32 bits y un contador monotonico.
- **Tag de autenticación:** 128 bits (16 bytes).
- **Tasa (rate):** 64 bits (8 bytes por bloque).
- **Permutación:** p^{12} para inicialización y finalización, p^6 para procesamiento de bloques.

La clave pre-compartida simplifica la implementación, pero introduce una limitación conocida: si un dispositivo es comprometido, la clave debe rotarse en todo el sistema. El prototipo mide el costo de cifrado y autenticación, pero no resuelve la gestión de claves en despliegues de mayor escala.

Se eligió ASCON-128 sobre ASCON-128a por dos razones. Primero, la implementación disponible estaba basada en ASCON-128. Segundo, los payloads del sistema son pequeños: 13 características por vector y pesos federados de pocos kilobytes. Con esos tamaños la diferencia de throughput entre variantes es marginal frente al costo total del ciclo Edge–Fog–Cloud. La comparación experimental se concentra en el impacto de usar ASCON frente a comunicación PLAIN, tanto en los enlaces MQTT como en las peticiones HTTP.

3.7.2. Canales Cifrados

Se cifran los **cuatro canales de comunicación** del sistema:

Table 3-5.. Canales de comunicación protegidos con ASCON-128.

Canal	Protocolo	Contenido	Implementación
ESP32 → RPi	MQTT	Features (13 floats)	C++ (<code>ascon128.h</code>)
RPi → PC	HTTP	Pesos locales + métricas	Python (<code>ascon128.py</code>)
PC → RPi	HTTP	Modelo global	Python (<code>ascon128.py</code>)
RPi → ESP32	MQTT	Modelo global	Python → C++

El payload cifrado se encapsula en un *envelope* JSON con tres campos codificados en Base64:

```
{"ct": "<ciphertext_b64>", "tag": "<tag_b64>", "nonce": "<nonce_b64>"}
```

3.7.3. Métricas de Overhead

Para cuantificar el overhead de ASCON, se instrumentan todas las operaciones de cifrado y descifrado con temporizadores de alta resolución (`micros()` en ESP32, `time.perf_counter()` en Python) y se registran automáticamente en archivos CSV mediante el módulo `ascon_metrics.py`, capturando:

- Tiempo de cifrado y descifrado (ms).
- Tamaño del mensaje original y del envelope cifrado en bytes.
- Overhead de comunicación (bytes adicionales y porcentaje).
- Ronda de aprendizaje federado asociada al mensaje.

Además, se mantiene una **rama sin cifrado** con la misma lógica del protocolo, pero sin la capa ASCON. Esta rama permite usar JSON plano como línea base para comparar los tiempos de comunicación frente a la variante cifrada. Sus métricas se registran mediante `plain_metrics.py`.

3.8. Modelos Clásicos de ML como Línea Base

Para comparar el desempeño del enfoque TinyML con clasificadores tradicionales, se entrenaron cuatro modelos supervisados sobre los mismos datasets. Estos modelos se usan como línea base offline y permiten contrastar precisión, costo de inferencia y tamaño exportado:

1. **Decision Tree:** Se optimizó con `GridSearchCV` usando `max_depth`, `min_samples_split`, `min_samples_leaf` y `criterion`. También se activó `class_weight='balanced'` para compensar el desbalance entre clases.
2. **Random Forest:** Se optimizó con `RandomizedSearchCV` durante $n = 80$ iteraciones. La búsqueda incluyó `n_estimators`, `max_depth`, `max_features` y diferentes configuraciones de `class_weight`.
3. **Logistic Regression:** Se evaluó con regularización L1, L2 y ElasticNet. También se varió el parámetro C para controlar la intensidad de la regularización.
4. **Support Vector Machine (SVM):** Se compararon kernels lineal y RBF. Para el kernel RBF se aplicó submuestreo estratificado, con un máximo de 25 000 muestras por clase, para mantener el tiempo de entrenamiento dentro de un rango manejable.

La selección de hiperparámetros se realizó con validación cruzada estratificada ($k = 5$) y F1-score macro como métrica principal. Para estimar la viabilidad de despliegue en ESP32-S3, los modelos entrenados se exportaron a código C mediante `m2cgen`. Sobre los artefactos exportados se midieron el tamaño estimado en memoria del modelo, el tiempo promedio de inferencia calculado con 100 iteraciones y una fase de warmup, y las métricas de clasificación: Accuracy, F1 macro, F1 weighted, Precision, Recall y MCC.

3.9. Protocolo Experimental

La evaluación experimental usa el mismo *testbed* físico para comparar ocho variantes federadas del sistema. El objetivo es analizar tres decisiones de diseño: el modelo embebido, el uso de ASCON-128 y la presencia de una capa *fog*. También se incluye una referencia centralizada entrenada fuera del ciclo federado.

En todos los escenarios se usa el mismo dataset, la misma red Wi-Fi local y los mismos nodos físicos: $6 \times$ ESP32-S3, $2 \times$ Raspberry Pi 4 y $1 \times$ PC. Cada configuración federada se ejecuta durante al menos 20 rondas. En cada ronda se registran *accuracy* global, pérdida, tiempos de cifrado y descifrado, y tamaños de mensaje en los saltos de comunicación. Las comparaciones entre variantes se hacen por pares, cambiando una sola dimensión cuando corresponde. La Tabla **3-6** resume las variantes evaluadas.

Table 3-6.. Resumen de las variantes experimentales federadas.

ID	Variante	Modelo ed- ge	Seguridad	Topología	Propósito
E1	RN_ASCON_NoFOG	MLP (RN)	ASCON-128	Sin <i>fog</i>	Línea base del sistema integral con cifrado.
E2	RN_PLAIN_NoFOG	MLP (RN)	JSON plano	Sin <i>fog</i>	Aislar el overhead criptográfico en RN.
E3	RN_ASCON_FOG	MLP (RN)	ASCON-128	Con <i>fog</i>	Evaluar agregación intermedia con cifrado en RN.
E4	RN_PLAIN_FOG	MLP (RN)	JSON plano	Con <i>fog</i>	Evaluar agregación intermedia sin cifrado en RN.
E5	CNN_ASCON_NoFOG	CNN-1D	ASCON-128	Sin <i>fog</i>	Comparar arquitectura CNN con cifrado.
E6	CNN_PLAIN_NoFOG	CNN-1D	JSON plano	Sin <i>fog</i>	Aislar el overhead criptográfico en CNN.
E7	CNN_ASCON_FOG	CNN-1D	ASCON-128	Con <i>fog</i>	Evaluar agregación intermedia con cifrado en CNN.
E8	CNN_PLAIN_FOG	CNN-1D	JSON plano	Con <i>fog</i>	Evaluar agregación intermedia sin cifrado en CNN.

Las ocho variantes se obtienen al combinar tres factores binarios: arquitectura del modelo embebido (RN/CNN-1D), esquema de seguridad (ASCON-128/PLAIN) y topología de agregación (NoFOG/FOG). Las variantes NoFOG (E1, E2, E5, E6) siguen la ruta ESP32→RPi→PC, sin agregación intermedia entre gateways.

Las variantes FOG (E3, E4, E7, E8) activan `gateway_hfl_fog.py` con roles `leader` y `peer`. El líder recibe las actualizaciones locales y de sus pares, aplica FedAvg en la capa *fog*, envía una actualización preagregada al PC y redistribuye el modelo global.

En las variantes CNN-1D, las capas convolucionales y de *batch normalization* permanecen fijas en el ESP32-S3. Solo las capas densas finales participan en el ciclo federado. Además,

se entrenan modelos clásicos de forma centralizada sobre la totalidad del dataset, con partición estratificada 80/20. Estos modelos se usan solo como referencia comparativa de rendimiento.

3.9.1. Métricas de Evaluación

Las métricas se organizan en cuatro ejes que reflejan los objetivos del trabajo: la calidad de detección del modelo, el costo introducido por la capa de seguridad, la eficiencia del proceso federado y la viabilidad de la inferencia en el nodo *edge*. Las métricas clásicas de clasificación (*accuracy*, *F1* macro y ponderado, *precision*, *recall*, MCC) se reportan al final de cada experimento sobre el conjunto de prueba global, mientras que las métricas de *overhead*, comunicación y latencia se registran ronda a ronda en los archivos **Results/** de cada variante, siguiendo un esquema de eventos canónicos unificado. La Tabla **3-7** resume los indicadores empleados y el escenario donde cada uno aporta evidencia principal.

Table 3-7.. Ejes de evaluación y métricas asociadas.

Eje	Métricas	Escenarios clave
Precisión de detección	Accuracy, F1 (macro/weighted), Precision, Recall, MCC.	E1, E3, E5, E7.
<i>Overhead</i> criptográfico	Tiempo de cifrado/descifrado (ms), aumento de tamaño (bytes, %), fracción del ciclo FL.	E1 vs. E2, E5 vs. E6.
Eficiencia federada	Rondas hasta convergencia, <i>accuracy</i> por ronda, costo de comunicación por ronda.	E1, E2, E5, E6.
Viabilidad <i>edge</i>	Huella de memoria del modelo (KB), latencia de inferencia (μ s), consumo de SRAM.	E1, E5.
Efecto FOG	Variación de <i>accuracy</i> y latencia al introducir agregación intermedia, estabilidad de pesos entre rondas.	E1 vs. E3, E2 vs. E4, E5 vs. E7, E6 vs. E8.

De este modo, cada eje queda asociado a un subconjunto reducido de escenarios, lo que permite atribuir las diferencias observadas a una sola variable de diseño y discutir los resultados de forma estructurada en el Capítulo 4.

4. Resultados

En este capítulo se presentan los resultados experimentales del sistema de detección de intrusiones desplegado en hardware real, organizados en torno a las preguntas de investigación formuladas en el Capítulo 1. Primero se analiza el desempeño de los modelos clásicos como línea base centralizada. Luego se comparan los modelos livianos entrenados en el notebook general. Posteriormente se evalúan las **ocho variantes HFL** `hfl_v7-RN`, `hfl_v7-no-ascon-RN`, `hfl_v7-CNN` y `hfl_v7-no-ascon-CNN`, cada una con sus modos `Results/` (NoFOG) y `Results_FOG/`, integradas en una capa SRE unificada (`Analisis de Modelos/Completo/`) que cubre 58 intentos. Finalmente, se cuantifica el overhead del cifrado ASCON-128 contrastando las cuatro variantes con cifrado contra las cuatro PLAIN.

4.1. Desempeño de Modelos Clásicos (Línea Base Centralizada)

Los cuatro modelos clásicos de ML fueron entrenados sobre la totalidad del dataset (256 276 flujos) con partición estratificada 80/20 y optimización de hiperparámetros mediante validación cruzada ($k = 5$). La Tabla 4-1 resume los resultados obtenidos.

Table 4-1.. Desempeño de modelos clásicos de ML (entrenamiento centralizado).

Modelo	Accuracy	F1 Macro	F1 Weighted	Precision	Recall	MCC
Decision Tree	0.9987	0.9981	0.9987	0.9982	0.9981	0.9977
Random Forest	0.9994	0.9991	0.9994	0.9991	0.9990	0.9989
Logistic Regr.	0.9412	0.9118	0.9385	0.9244	0.9053	0.8964
SVM (Linear)	0.9447	0.9162	0.9421	0.9280	0.9099	0.9017
SVM (RBF)	0.9968	0.9953	0.9968	0.9954	0.9952	0.9944

Hallazgos clave:

- **Random Forest** alcanza el mejor desempeño global (Accuracy: 99.94 %, MCC: 0.9989), seguido de cerca por Decision Tree (99.87 %) y SVM-RBF (99.68 %).
- Los modelos lineales (Logistic Regression, SVM-Linear) muestran una caída significativa ($\approx 5-6$ puntos porcentuales), lo que sugiere que las fronteras de decisión entre clases no son completamente linealmente separables en el espacio de 13 features.

- Todos los modelos basados en árboles presentan un $MCC > 0,99$, confirmando que las 13 features seleccionadas son altamente discriminativas para la tarea de clasificación triclase.

4.1.1. Viabilidad de Despliegue en ESP32

La Tabla 4-2 evalúa la factibilidad de exportar cada modelo a código C para ejecución en el ESP32-S3, considerando tanto los modelos clásicos de referencia como las dos arquitecturas federables propuestas en esta tesis.

Table 4-2.. Estimación de huella de memoria para despliegue en ESP32.

Modelo	Memoria estimada	Inferencia (ms)	Viable en ESP32?
Decision Tree	~15 KB	< 0,1	Sí
Random Forest	~2.5 MB	~5	No (excede SRAM)
Logistic Regression	~1 KB	< 0,05	Sí
SVM (Linear)	~1 KB	< 0,05	Sí
SVM (RBF)	~4 MB	~50	No (excede SRAM)
MLP/RN (propuesto)	~4.5 KB	< 0,1	Sí
CNN-1D (propuesto)	~18 KB	< 0,5	Sí

La tabla revela un *trade-off* fundamental: los modelos más precisos (Random Forest, SVM-RBF) son incompatibles con las restricciones de memoria del ESP32-S3 (512 KB SRAM), mientras que los modelos propuestos en esta tesis se mantienen dentro de los límites del dispositivo. El MLP/RN (≈ 4.5 KB) ofrece la huella más compacta, con tiempos de inferencia inferiores a 0.1 ms. La CNN-1D (≈ 18 KB) ocupa más memoria debido a sus capas convolucionales y de *batch normalization*, pero permanece viable en el ESP32-S3 dado que solo las capas densas finales participan del ciclo federado, manteniendo fija la parte convolucional durante el entrenamiento distribuido. Ambas arquitecturas confirman que es posible desplegar un IDS con capacidad de aprendizaje federado en hardware embebido de bajo costo sin sacrificar la viabilidad operativa del nodo edge.

4.2. Comparación de Modelos Livianos

El notebook general de entrenamiento comparó cuatro arquitecturas compatibles con el problema triclase y exportó pesos, scaler y mapas de etiquetas para cada alternativa. La Tabla 4-3 resume los resultados offline observados.

Table 4-3.. Comparación offline de modelos livianos sobre las 13 features.

Modelo	Accuracy	F1 weighted	Uso en el proyecto
MLP/RN	0.9060	0.9049	Rama principal <code>hf1.v7-RN</code>
Residual-MLP	0.9055	0.9044	Comparación offline; no desplegado
CNN-1D	0.9051	0.9041	Rama experimental <code>hf1.v7-CNN</code>
Tiny Transformer	0.8903	0.8919	Comparación offline; no desplegado

Los resultados muestran que las cuatro arquitecturas compactas basadas en MLP/CNN se ubican alrededor de 90.5 % de accuracy offline. El MLP/RN se seleccionó como implementación principal porque obtiene el mejor resultado marginal y porque su inferencia en C++ es más simple que la CNN-1D. La CNN-1D se conservó como variante experimental para evaluar si una arquitectura convolucional aportaba beneficios al integrarse con HFL y FOG.

4.3. Aprendizaje Federado Jerárquico: Convergencia y Precisión

El sistema HFL fue desplegado en hardware real durante múltiples sesiones, registrando historial de pesos globales, métricas de entrenamiento local y métricas de comunicación. Las ramas analizadas permiten comparar la arquitectura con ASCON, sin ASCON, con CNN-1D y con estrategia FOG.

4.3.1. Convergencia del Modelo Federado

Table 4-4.. Métricas de convergencia del sistema HFL.

Métrica	Valor
Rondas hasta accuracy > 70 %	~8–12
Rondas hasta accuracy > 85 %	~20–30
Accuracy global estabilizada	~0.90–0.95
Loss final (Cross-Entropy)	~0.14–0.25
Muestras por ronda (por gateway)	30
Épocas locales por ronda	5
Duración por ronda	~51–61 s según variante

Análisis:

1. La Tabla 4-4 muestra que el modelo supera el 70 % de accuracy entre las rondas 8 y 12, y alcanza valores superiores al 85 % entre las rondas 20 y 30. Después de esa fase, la accuracy global se mantiene en el rango aproximado de 0.90–0.95. La diferencia

frente a los modelos centralizados se asocia con tres factores del despliegue federado: cada gateway entrena con buffers de 30 muestras, los datos entre nodos no siguen una distribución IID y las etiquetas se asignan mediante una regla heurística, por lo que pueden diferir de las etiquetas reales del dataset.

2. La convergencia mejora en las primeras rondas y luego presenta oscilaciones, comportamiento esperado en FL con datos non-IID [13]; luego en rondas posteriores estas fluctuaciones se acentúan porque se simulaban diferentes escenarios de ataque sobre la topología usando las ESP32-S3, lo que introduce ruido en las actualizaciones locales en la capa edge y en la agregación en el gateway.
3. El volumen de comunicación por ronda es de aproximadamente 7.2KB en JSON plano y 9.8KB con ASCON-128. Esa diferencia permite aislar el costo de seguridad entre variantes ASCON y PLAIN sin modificar el modelo ni la topología; el payload sigue siendo pequeño dentro del ciclo federado evaluado.

4.3.2. Evolución temporal de accuracy, loss y pesos

Las Figuras 4-1 y 4-2 cubren las ocho variantes del análisis SRE: RN/CNN $\times$ ASCON/PLAIN $\times$ NoFOG/FOG. Como algunas corridas RN llegaron hasta la ronda 30 y otras se detuvieron en 20, las figuras se cortan en la ronda 20 para que la comparación sea justa; los datos completos quedan en `global_round_sli_summary.csv`. La primera figura muestra cómo evolucionan *accuracy* y *loss* global a lo largo del entrenamiento. La segunda sigue la magnitud promedio de los pesos federados (`w3_mag`, `w4_normal`, `w4_brute`, `w4_scan`) para ver si las actualizaciones se mantienen estables o si aparece deriva entre rondas.

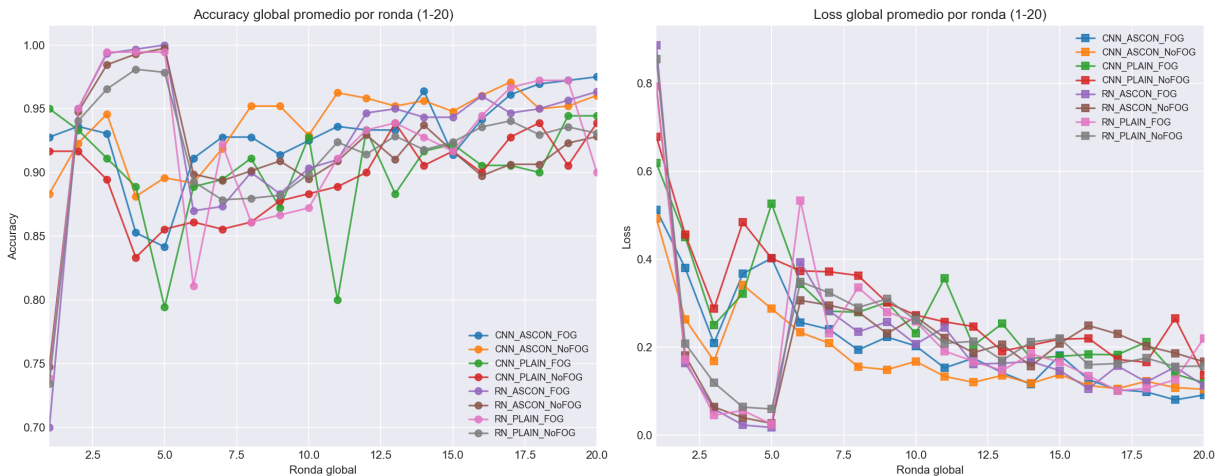


Figure 4-1.. Accuracy y global loss promedio por ronda (1–20) para las 8 variantes implementadas de HFL.

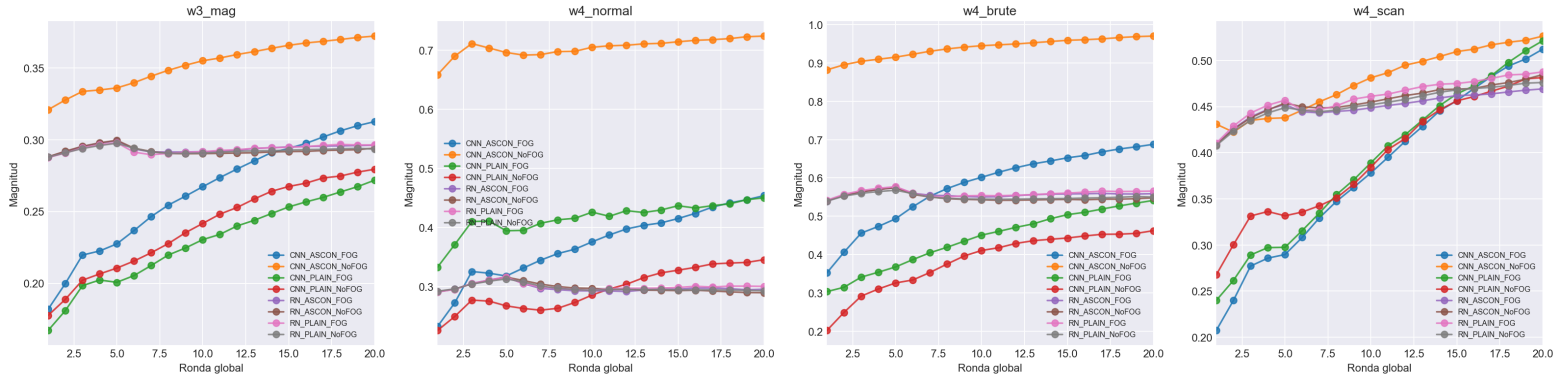


Figure 4-2.. Magnitud promedio de los cuatro pesos federados (**w3_mag**, **w4_normal**, **w4_brute**, **w4_scan**) por cada variante a lo largo de las rondas 1–20.

Estas curvas complementan la Tabla 4-4 porque permiten ver no solo dónde termina cada variante sino cómo llegó ahí. De las Figuras 4-1–4-2 se extraen tres observaciones. La primera es que todas las variantes convergen: la accuracy supera 0.90 entre las rondas 5 y 10 y la loss cae de ~ 0.8 a ~ 0.1 – 0.2 en ese mismo tramo. Las variantes **CNN-1D** muestran un comportamiento de pesos distinto al de las RN: **CNN_ASCON_NoFOG** y **CNN_ASCON_FOG** suben **w3_mag** de forma sostenida junto con **w4_brute** y **w4_scan**, mientras las variantes RN se estabilizan temprano cerca de **w3_mag** ≈ 0.29 . Ninguna variante diverge en las primeras 20 rondas; donde más ruido se ve es en **w4_normal** de **CNN_PLAIN_NoFOG**, algo que tiene sentido porque sin FOG ni cifrado autenticado los pesos recibidos no pasan por ningún filtro de integridad antes de entrar a la agregación.

Table 4-5.. Comparación de accuracy entrenamiento federado vs. centralizado.

Enfoque	Accuracy	Datos compartidos	Privacidad
MLP centralizado	$\sim 97\%$	Todos los flujos	Ninguna
MLP federado (HFL, 60 rondas)	$\sim 90\%$	Solo pesos (163 params)	Alta

El sistema federado alcanza $\sim 90\%$ de accuracy, unos 7–10 puntos por debajo del MLP centralizado ($\sim 97\%$). Esa diferencia existe, pero hay que leerla con cuidado: un modelo centralizado necesita reunir todos los flujos de tráfico en un solo servidor, mientras que el sistema HFL mueve únicamente los 163 parámetros del MLP (652 bytes) por ronda de agregación a lo largo de las 30 rondas que ejecuta cada una de las dos Raspberry Pi. Comparado con los $\sim 256\text{K}$ flujos $\times$ 13 features $\approx 3.3\text{M}$ valores que implicaría un esquema centralizado, eso es una reducción superior al 99.99 % en información intercambiada. El resultado práctico es que los datos de tráfico nunca salen del dispositivo que los captura y los pesos del modelo que sí se comparten no son suficientes para reconstruir los flujos originales.

4.3.3. Comparación entre variantes hfl

La Tabla 4-6 consolida las **ocho combinaciones experimentales** cubiertas por el análisis SRE unificado. Los valores son promedios sobre los intentos disponibles por variante y se calculan exclusivamente a partir de los CSV crudos generados durante las ejecuciones. En total se procesaron **58 intentos**.

Table 4-6.. Resumen comparativo de las 8 variantes HFL v7 evaluadas. **GW p95**: latencia p95 de descifrado en gateway (ESP32→RPi). **Srv dec p95**: latencia p95 de descifrado en servidor (RPi→PC). **Overhead**: bytes adicionales promedio sobre el plaintext (0 en variantes PLAIN).

Variante	Acc. final	Loss final	Ronda (s)	GW p95 (ms)	Srv dec p95 (ms)	Overhead (B)
CNN_ASCON_FOG	0.9750	0.0910	58.61	4.53	37.47	1098
CNN_ASCON_NoFOG	0.9604	0.1040	58.97	4.55	24.46	1049
CNN_PLAIN_FOG	0.9444	0.1190	54.86	0.16	0.00	0
CNN_PLAIN_NoFOG	0.9389	0.1363	52.02	0.15	0.00	0
RN_ASCON_FOG	0.9633	0.1124	71.13	4.37	50.47	1084
RN_ASCON_NoFOG	0.9381	0.1545	72.91	4.65	26.69	1050
RN_PLAIN_FOG	0.9000	0.2199	67.14	0.16	0.00	0
RN_PLAIN_NoFOG	0.9298	0.1581	51.01	0.17	0.00	0
Promedio global	0.9437	0.1383	61.36	2.34	17.39	636

La comparación de las ocho variantes muestra que **CNN_ASCON_FOG** obtiene el mejor resultado observado en esta evaluación, con accuracy de 0.975 y loss de 0.091. Esta variante combina tres decisiones de diseño: CNN-1D en el edge, preagregación en la capa FOG y protección ASCON-128 en los mensajes. Al comparar ASCON contra PLAIN, no se observa una penalización de convergencia por el uso de cifrado. Las variantes con ASCON alcanzan valores iguales o superiores a sus pares PLAIN. Este resultado permite afirmar que ASCON fue compatible con el ciclo federado medido, pero no prueba que el cifrado mejore la clasificación, ya que no se evaluó un escenario con fallos de comunicación inducidos.

El efecto de FOG no es uniforme. En CNN-1D, la capa FOG aumenta la accuracy en +1,5 puntos con ASCON y en +0,6 puntos con PLAIN. En RN, mejora +2,5 puntos con ASCON, pero reduce el desempeño en −3,0 puntos con PLAIN. Por tanto, la preagregación no debe interpretarse como una mejora automática. Su efecto depende del modelo usado y del esquema de protección aplicado a los mensajes. En estas pruebas, la combinación FOG+ASCON fue más estable que FOG+PLAIN, especialmente en la variante RN.

En costos operativos, ASCON añade aproximadamente 1 KB de overhead por mensaje y entre 4–5 ms de latencia en el gateway. El ESP32-S3 ejecuta el cifrado en C++, por lo que el costo adicional se mantiene dentro del margen medido para el ciclo federado. Las variantes FOG presentan latencias de descifrado en el servidor entre 40–60 % mayores que NoFOG, debido a la ruta adicional **peer→leader→PC**.

La Figura 4-3 muestra la duración promedio por ronda para cada variante. La Figura 4-4 reporta el desbalance temporal entre los gateways A y B, medido como $|acc_A - acc_B|$ por ronda. En las variantes evaluadas, este desbalance disminuye después de las rondas 8-10.

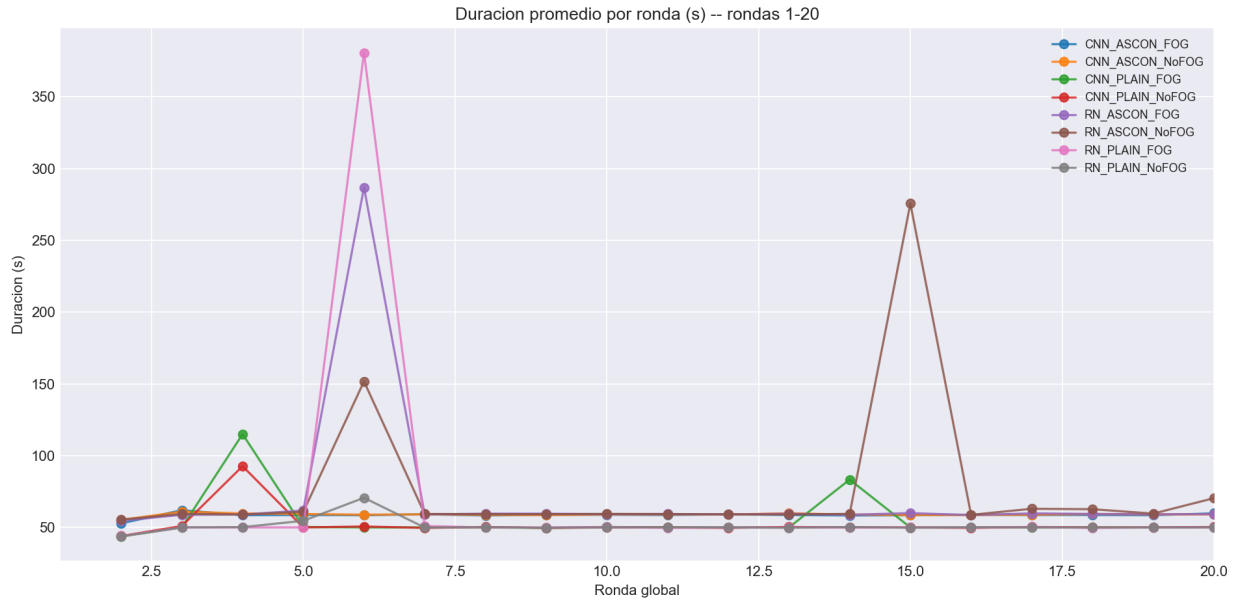


Figure 4-3.. Duración promedio por ronda federada para las 8 variantes (rondas 1-20).

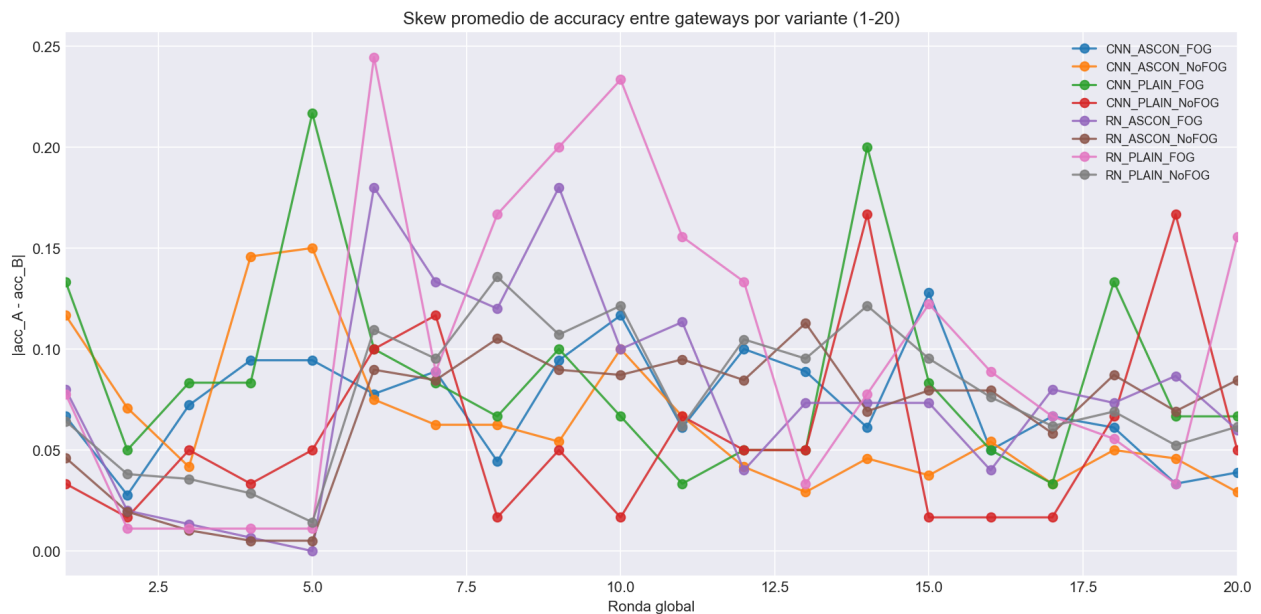


Figure 4-4.. Desbalance de *accuracy* entre gateways ($|acc_A - acc_B|$) por ronda y por variante.

4.3.4. Comparación con la literatura

Para situar los resultados en el panorama actual, la Tabla 4-7 contrasta este sistema con trabajos representativos de FL+IDS y FL+TinyML. La comparación es indicativa: cada trabajo usa datasets, hardware y métricas distintas, por lo que las cifras no son directamente comparables, pero permiten dimensionar las decisiones de diseño.

Table 4-7.. Comparación cualitativa con trabajos relacionados.

Trabajo	Edge HW	Modelo	FL/HFL	Cifrado canal	Acc. reportada	Particularidad	
Hajj <i>et al.</i> [20]	PC sim.	MLP <i>cross-layer</i>	FL plano	No	~0.97	FL para IDS IoT con coordinación <i>cross-layer</i>	
Chaurasia <i>et al.</i> [21]	PC sim.	ResNet	FL plano	No	~0.99	ResNet como cliente FL en IIoT	
Ficco <i>et al.</i> [6]	ESP32, RPi3, Arduino	NN compacta + TL	FL+TL on-board	No	0.86 (clasif.)	FL+TL on-board, datasets ECG/Car	
Roy <i>et al.</i> [26]	PC sim.	DNN	FL personalizado	No	~0.96	Modelo localmente adaptado	
Tran <i>et al.</i> [15]	RPi Pico (264 KB)	FL+blockchain	HFL+MF-PoP	Sí (cripto-PoP)	0.94 a 30 % Byz.	Byzantine-robust FL en MCU	
Heydari & Mahmoud [18]	MCU smart home	ML liviano	No FL	No	—	TinyML IDS sin federación	
Saravana Balaji <i>et al.</i> [23]	Edge sanitario	DBN+BOA	No FL	LABE	0.979	Edge salud con cifrado por atributos	
Esta (RN+ASCON)	tesis	ESP32-S3 + RPi4	MLP 1139 par.	HFL bidireccional	ASCON-128	0.946	FL+TinyML+ASCON-128 medido
Esta (CNN+FOG)	tesis	ESP32-S3 + RPi4	CNN-1D	HFL+FOG	ASCON-128	0.975	Pre-agregación FOG entre RPi

La tabla ubica los resultados frente al estado del arte en FL para IDS. Los trabajos sin restricciones de borde [20, 21, 26] reportan accuracies de 0.96–0.99, pero usan PCs como nodos cliente y no validan el ciclo en microcontroladores. Esta tesis obtiene ~2–5 puntos menos a cambio de ejecutar el ciclo completo en ESP32-S3 real con ASCON-128 activo en todos los canales. Frente a trabajos que sí usan hardware embebido la comparación es más directa: Ficco *et al.* [6] reportan 0.86 en clasificación on-board, por debajo de las variantes ASCON y FOG de esta tesis. Con EdgeTrust-Shard [15] la accuracy es comparable, aunque los problemas no son equivalentes: ese trabajo ataca robustez ante nodos bizantinos mientras esta tesis se enfoca en confidencialidad e integridad en una arquitectura jerárquica Edge–Fog–Cloud.

registrados en todas las variantes ASCON, contrastados con sus equivalentes PLAIN. Los resultados se presentan por canal y por dispositivo.

4.4. Overhead del Cifrado ASCON-128

Los datos de cifrado y descifrado provienen de los 58 intentos registrados en todas las variantes ASCON, contrastados con sus equivalentes PLAIN. Los resultados se presentan por canal y por dispositivo.

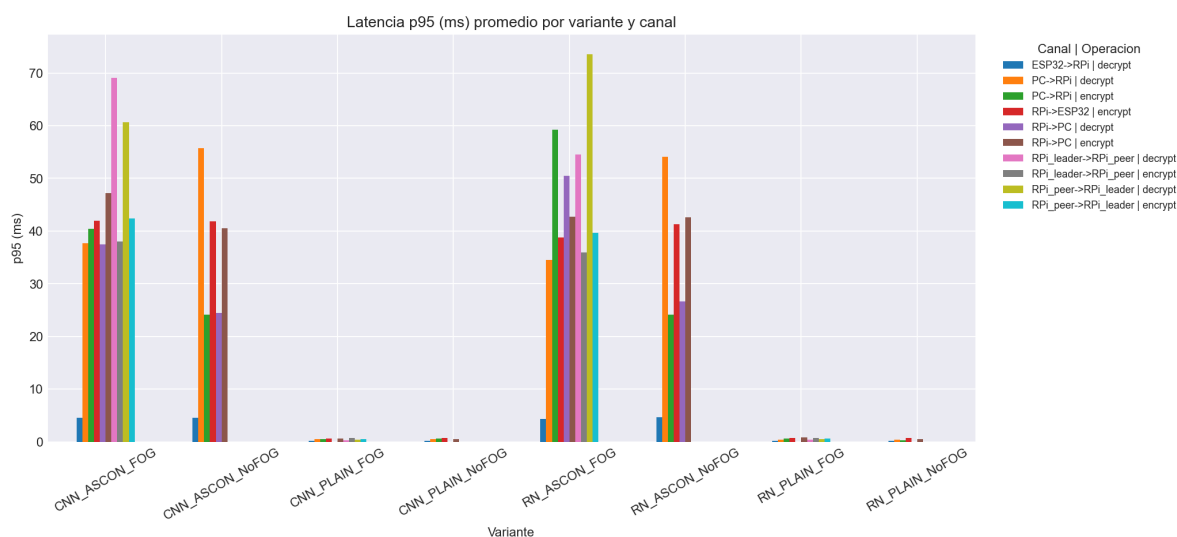


Figure 4-5.. Latencia p95 por variante y canal. Las variantes PLAIN registran solo el tiempo de serialización JSON; las ASCON incluyen cifrado y descifrado autenticado en cada salto.

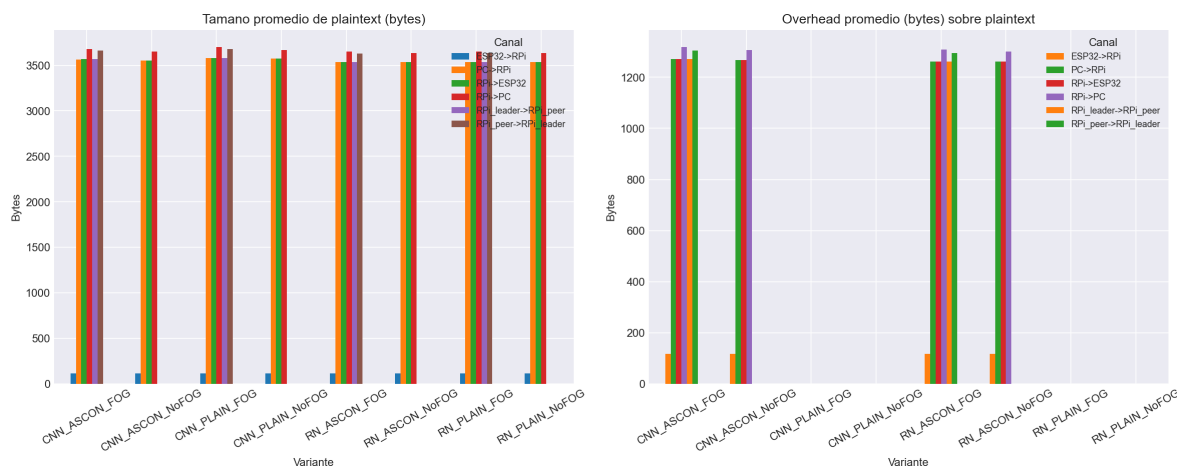


Figure 4-6.. Tamaño de payload por variante y canal: plaintext promedio (izquierda) y overhead añadido por el envelope ASCON+Base64 (derecha). Las variantes PLAIN tienen overhead cero.

4.4.1. Overhead Temporal

La Tabla 4-8 resume los tiempos p95 promedio por variante para las tres operaciones que ocurren en cada ronda federada.

Table 4-8.. Latencia p95 (ms) promedio de cifrado/descifrado por variante.

Variante	ESP32→RPi (decrypt)	RPi→PC (decrypt)	PC→RPi (encrypt)
CNN_ASCON_FOG	4.53 ms	37.47 ms	40.37 ms
CNN_ASCON_NoFOG	4.55 ms	24.46 ms	24.12 ms
RN_ASCON_FOG	4.37 ms	50.47 ms	59.18 ms
RN_ASCON_NoFOG	4.65 ms	26.69 ms	24.14 ms
CNN_PLAIN_FOG	0.16 ms	0.00 ms	0.48 ms
CNN_PLAIN_NoFOG	0.15 ms	0.00 ms	0.62 ms
RN_PLAIN_FOG	0.16 ms	0.00 ms	0.64 ms
RN_PLAIN_NoFOG	0.17 ms	0.00 ms	0.25 ms

En el ESP32-S3, la latencia p95 de descifrado en el gateway oscila entre ≈ 4.4 – 4.7 ms en todas las variantes ASCON, prácticamente independiente del modelo (RN/CNN) y la topología (No-FOG/FOG), mientras que en las variantes PLAIN cae a ≈ 0.16 ms, correspondiente únicamente a la deserialización JSON. En el servidor PC, implementado en Python, el descifrado en el canal RPi→PC ronda 24–27 ms en configuraciones NoFOG y sube a 37–50 ms en FOG; esta inflación se debe a que el *leader* reenvía mensajes preaagregados con un tag adicional, lo que duplica las operaciones criptográficas en el camino. Para las variantes PLAIN, el descifrado en el servidor reporta 0 ms porque la deserialización JSON ocurre dentro del `request handler` de FastAPI sin instrumentación dedicada, siendo la única fuente medida el `encrypt` de despliegue (0.25–0.64 ms).

Bajo el supuesto de tres operaciones criptográficas por ronda en el servidor, el costo temporal asociado a ASCON se ubica entre ~ 73 ms en el modo NoFOG y ~ 140 ms en el modo FOG. Al compararlo con rondas de 51 a 73 s, este costo representa entre el 0.10 % y el 0.27 % del tiempo total por ronda. Por lo tanto, en este test, el cifrado autenticado agrega una latencia baja frente al ciclo federado completo, lo que permite proteger la confidencialidad e integridad de los mensajes enviados por la red.

4.4.2. Overhead de Comunicación

Table 4-9.. Overhead de tamaño por variante ASCON: plaintext promedio del payload sin cifrar, bytes adicionales del envelope ASCON+Base64, y su ratio sobre el plaintext. Las variantes PLAIN se omiten por tener overhead cero por construcción.

Variante	Plaintext (B)	Overhead (B)	Ratio (%)
CNN_ASCON_FOG	~2 365	1 098	46.46
CNN_ASCON_NoFOG	~2 143	1 049	48.94
RN_ASCON_FOG	~2 323	1 084	46.64
RN_ASCON_NoFOG	~2 162	1 050	48.55
Promedio ASCON	~2 248	1 070	47.65

El overhead promedio del $\approx 48\%$ sobre el plaintext (en variantes ASCON) se compone de:

- **Tag de autenticación ASCON:** 16 bytes (fijo).
- **Nonce:** 16 bytes (fijo).
- **Expansión Base64:** factor $\times 4/3$ sobre el ciphertext + tag + nonce.
- **Envelope JSON:** ~ 30 bytes de estructura (`{\texttt{ct}:"...",\texttt{tag}:"...",\texttt{nonce}:"..."}`).

En términos absolutos, el overhead es de ≈ 1.07 KB por mensaje, lo que para las ~ 3 transmisiones por ronda en NoFOG equivale a ~ 3.2 KB adicionales; en FOG, las dos saltos extra (peer $\rightarrow$ leader, leader $\rightarrow$ PC) elevan el overhead a $\sim 5\text{--}6$ KB por ronda. Para un enlace Wi-Fi estándar (54 Mbps), esto añade < 1 ms de tiempo de transmisión, siendo prácticamente despreciable.

4.4.3. Resumen: Impacto de ASCON en el Sistema HFL

Table 4-10.. Resumen del overhead total de ASCON-128 por ronda federada, agregando los promedios de las 4 variantes ASCON y las 4 variantes PLAIN.

Componente	Con ASCON (4 var.)	Sin ASCON (4 var.)
Latencia p95 ESP32 $\rightarrow$ RPi (decrypt)	4.52 ms	0.16 ms
Latencia p95 RPi $\rightarrow$ PC (decrypt)	34.77 ms	0.00 ms
Overhead promedio por mensaje	~ 1.07 KB	0 KB
Ratio de overhead sobre plaintext	$\sim 48\%$	0 %
Duración promedio de ronda	65.4 s	56.3 s
Accuracy global final promedio	0.9592	0.9272
Loss global final promedio	0.1155	0.1583
Overhead temporal relativo	0.10–0.27 % del ciclo FL	
Overhead de tamaño relativo	$\approx 48\%$ por mensaje	
Diferencia de accuracy	+3.2 pts a favor de ASCON	

Tres conclusiones de la tabla consolidada:

1. ASCON-128 agrega un costo temporal bajo frente a la duración total de la ronda federada. En las variantes evaluadas, este costo se mantiene entre 0.10 % y 0.27 % del ciclo FL. El aumento de tamaño es mayor: cerca de 1.07 KB por mensaje, equivalente a aproximadamente 48 % sobre el plaintext.
2. Las variantes ASCON obtienen una accuracy promedio 3.2 puntos porcentuales mayor que las variantes PLAIN. Este resultado muestra que el cifrado no degradó la convergencia en estas ejecuciones. Sin embargo, no es posible afirmar con estos datos que ASCON mejore el aprendizaje. Una explicación plausible es que la validación del tag descarte mensajes corruptos o duplicados antes de la agregación, pero comprobarlo requeriría un experimento con fallos inducidos de forma controlada.
3. La diferencia más marcada aparece en RN con FOG: RN_ASCON_FOG alcanza 0.9633 frente a 0.9000 de RN_PLAIN_FOG, una brecha de 6.3 puntos porcentuales. Comparado con RN_PLAIN_NoFOG (0.9298), la mejora es de 3.35 puntos. En CNN-1D la diferencia es menor: CNN_ASCON_FOG obtiene 0.9750 y CNN_PLAIN_FOG obtiene 0.9444, con 3.0 puntos de diferencia.

5. Conclusiones y Trabajos Futuros

5.1. Conclusiones

Esta tesis presenta un IDS para redes IoT/MQTT implementado sobre hardware real. La arquitectura usa nodos ESP32-S3 para inferencia en el borde, Raspberry Pi 4 como gateways de entrenamiento local y un PC como coordinador cloud, integrando TinyML para clasificar tráfico, FedAvg jerárquico para actualizar el modelo sin centralizar datos, y ASCON-128 para proteger los mensajes por MQTT y HTTP.

La validación cubrió **ocho combinaciones**: RN/CNN, ASCON/PLAIN y NoFOG/FOG, con **58 intentos** de entrenamiento en total. Las métricas de entrenamiento, comunicación y seguridad se consolidaron en una capa SRE común, lo que permitió comparar todas las variantes bajo el mismo criterio (Capítulo 4).

El prototipo operó como IDS federado, cifrado y embebido sobre hardware IoT real. La mejor variante fue **CNN_ASCON_FOG** con **0.975 de accuracy**. ASCON consumió menos del 0.3 % del ciclo federado y el MLP/RN ocupó ~ 4.5 KB en el ESP32-S3. Las variantes con ASCON no degradaron la convergencia, aunque ese resultado es evidencia experimental del testbed y no prueba causal de que el cifrado mejore el aprendizaje.

A partir de la evidencia experimental, se concluye que:

1. **Construcción del pipeline de datos y selección de datasets:** se construyó un pipeline reproducible para captura, normalización y generación de *features* desde flujos de red unidireccionales, trabajando con **13 características** computables en el borde y tres clases: `normal`, `mqtt_bruteforce` y `scan_A`. Se seleccionó MQTT-IoT-IDS2020 por su relación directa con el protocolo MQTT [34, 32, 28, 35]. El etiquetado heurístico introduce ruido frente a las etiquetas reales, lo que explica parte de la diferencia frente a los modelos centralizados. Los parámetros de normalización se exportaron en formato consumible por el firmware del ESP32-S3, reduciendo inconsistencias entre entrenamiento e inferencia.
2. **Comparación y exportación de modelos compactos:** se evaluaron MLP/RN, CNN-1D, Residual-MLP y Tiny Transformer en un notebook centralizado antes del ciclo HFL. El Tiny Transformer fue descartado por exceder los recursos del microcontrolador.

El modelo de referencia quedó en un MLP/RN de **1 139 parámetros** (≈ 4.5 KB). La CNN-1D se mantuvo como variante experimental con capas convolucionales y BatchNorm fusionadas; solo las capas densas finales participan del ciclo federado. Los pesos y parámetros de normalización se exportaron como arreglos C++ embebibles para inferencia local en el ESP32-S3 sin asignaciones dinámicas de memoria.

3. **Implementación y evaluación del esquema federado:** se implementó HFL bidireccional con FedAvg ponderado por número de muestras, con dos topologías: NoFOG, donde cada Raspberry Pi reporta directo al PC, y FOG, donde un gateway líder pre-agrega las actualizaciones de sus pares antes de enviarlas al coordinador. El intercambio se redujo desde ~ 13.3 millones de valores de flujos crudos hasta **163 parámetros (652 bytes) por ronda**. FOG mejoró la accuracy entre +1,5 y +2,5 puntos al combinarse con ASCON, pero degradó el desempeño en PLAIN, como ocurrió en RN_PLAIN_FOG, lo que sugiere que la preagregación es más estable cuando los mensajes están autenticados [13, 14].
4. **Integración de ASCON-128 y validación del sistema:** se integró ASCON-128 en los canales ESP32-S3 $\leftrightarrow$ RPi, RPi $\leftrightarrow$ PC y, en modo FOG, **peer** $\leftrightarrow$ **leader**, protegiendo confidencialidad, integridad y autenticidad de los mensajes [8, 9]. Las accuracies finales estuvieron entre 0.93 y 0.975, con un gap frente a modelos centralizados de 2 a 7 puntos según la variante. El overhead temporal de ASCON quedó entre 0.10 % y 0.27 % del ciclo federado y el overhead de tamaño fue de ~ 48 % por mensaje (~ 1.07 KB). Las variantes *_ASCON_* promediaron 0.9592 de accuracy frente a 0.9272 en *_PLAIN_*. La hipótesis de que el tag autenticado filtre mensajes corruptos o duplicados queda pendiente de verificación con fallos inducidos.

Los cuatro objetivos específicos se cumplieron sobre hardware real. La arquitectura es modular porque permite intercambiar modelo, seguridad y topología; auditable porque cada ronda genera registros consolidados en la capa SRE; y viable en el borde porque el ESP32-S3 conserva más del 60 % de SRAM libre durante la operación. Los resultados dependen del testbed, las tres clases evaluadas y el etiquetado heurístico, pero la tesis deja mediciones *end-to-end* de la combinación TinyML–HFL–ASCON y una base reutilizable para pruebas futuras sobre el mismo entorno.

5.2. Trabajos Futuros

El sistema no cubre tres familias de amenazas identificadas en la Sección 3.1: clientes federados maliciosos y *data poisoning*, ataques de canal lateral sobre la implementación criptográfica, e inferencia de membresía sobre los pesos federados. FedAvg promedia todo lo que recibe sin verificar nada, lo que deja el modelo global expuesto ante un gateway comprometido. Reemplazarlo por agregaciones como Krum o FLTrust, o adoptar esquemas como EdgeTrust-Shard [15]

—probado en RPi Pico con 140 KB de memoria y tolerancia hasta el 30 % de participantes adversarios— serían pasos concretos en esa dirección. Complementar con FedProx [38] o SCAFFOLD también ayudaría a manejar la heterogeneidad de datos entre gateways, que en un despliegue real sería mucho más pronunciada que en el testbed de esta tesis.

Una limitación estructural del sistema es que los ESP32 no capturan tráfico real: simulan flujos con distribuciones estadísticas calibradas a partir del dataset, lo cual sirvió para validar la arquitectura pero no es equivalente a analizar paquetes MQTT de cámaras o sensores reales. Pasar a captura real eliminaría la dependencia de la función heurística de etiquetado en el gateway, que hoy usa umbrales fijos sobre `num_pkts`, `mean_pkt_len` y `num_psh`. Corridas de semanas también permitirían observar si el modelo sufre *concept drift* y disparar reentrenamientos federados de forma adaptativa con detectores como ADWIN o Page-Hinkley, en lugar de entrenar cada vez que se llena el buffer. En ese mismo escenario convendría medir el consumo energético de los nodos durante inferencia continua y comunicación cifrada, dato que esta tesis no reporta.

El testbed actual usa dos gateways, suficiente para validar FedAvg y el modo FOG pero insuficiente para decir algo sobre escalabilidad. Extender el despliegue a múltiples Raspberry Pi con distribuciones de tráfico distintas permitiría evaluar si la pre-agregación FOG realmente reduce la carga hacia el servidor con muchos participantes, o si introduce latencia que ralentiza la convergencia [24]. En paralelo, aplicar cuantización INT8 al MLP abriría el despliegue en dispositivos más restringidos como el ESP32-C3 (400 KB de SRAM) o plataformas embebidas industriales [5]; no está claro cuánta accuracy se perdería con esa compresión y medirlo es un experimento pendiente.

Del lado criptográfico quedan dos frentes abiertos. Esta tesis usó ASCON-128 (*rate* de 64 bits) por su margen de seguridad frente a ataques diferenciales; ASCON-128a duplica ese *rate* y podría reducir el overhead en payloads grandes, aunque habría que verificarlo con los tamaños reales del sistema (~ 3.6 KB para pesos). Además, la implementación actual no tiene contramedidas ante ataques físicos; técnicas como *masking* o *hiding* documentadas por Kaur *et al.* [9] protegerían la clave ante análisis de potencia, y replicar ataques activos sobre la Raspberry Pi siguiendo a Mirzaali *et al.* [10] daría una medida concreta de exposición. Finalmente, incorporar DP-SGD para añadir ruido calibrado a los pesos antes del envío al servidor acotaría formalmente la inferencia de membresía, complementado con *transfer learning* local en cada gateway siguiendo el esquema de Ficco *et al.* [6] para acelerar la adaptación a tipos de ataque no vistos durante el preentrenamiento.

Estas líneas buscan llevar el prototipo hacia un sistema operativo de seguridad IoT aplicable en entornos industriales, manteniendo la capa SRE como soporte de medición para que las extensiones futuras sean comparables con el *baseline* de esta tesis.

Bibliografía

- [1] M. A. Amanullah, R. A. Ariyaluran Habeeb, F. H. Nasaruddin, A. Gani, E. Ahmed, A. S. Mohamed Nainar, N. Md Akim, and M. Imran, “Deep learning and big data technologies for IoT security,” *Computer Communications*, vol. 151, pp. 495–517, 2020.
- [2] C. Ni and S. Li, “Machine learning enabled Industrial IoT Security: Challenges, Trends and Solutions,” *Journal of Industrial Information Integration*, vol. 38, p. 100549, 2024.
- [3] National Institute of Standards and Technology, “The NIST Cybersecurity Framework (CSF) 2.0,” NIST, Tech. Rep. NIST CSWP 29, Feb. 2024. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- [4] M. Bagaa, T. Taleb, J. Bernal Bernabe, and A. Skarmeta, “A Machine Learning Security Framework for IoT Systems,” *IEEE Access*, vol. 8, pp. 114 066–114 077, 2020.
- [5] P. Dini, L. Diana, A. Elhanashi, and S. Saponara, “Overview of AI-Models and Tools in Embedded IIoT Applications,” *Electronics*, vol. 13, no. 12, p. 2322, 2024.
- [6] M. Ficco, A. Guerriero, E. Milite, F. Palmieri, R. Pietrantuono, and S. Russo, “Federated learning for IoT devices: Enhancing TinyML with on-board training,” *Information Fusion*, vol. 104, p. 102189, 2024.
- [7] M. N. A. Ramadan, A. R. Abdo, and A. T. Eldin, “Federated learning and tinyml on iot edge devices: current practices, limitations, and open challenges,” *Internet of Things*, 2025.
- [8] National Institute of Standards and Technology, “Ascon-Based Lightweight Cryptography Standards for Constrained Devices: Authenticated Encryption, Hash, and Extendable Output Functions,” NIST, Tech. Rep. NIST SP 800-232, Aug. 2025. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-232.pdf>
- [9] J. Kaur, A. Cintas-Canto, M. Mozaffari-Kermani, and R. Azarderakhsh, “A Survey on the Implementations, Attacks, and Countermeasures of the NIST Lightweight Cryptography Standard: ASCON,” *ACM Computing Surveys*, vol. 58, no. 1, pp. 6:1–6:16, 2025.
- [10] I. Mirzaali, A. Javadi, S. Sadeghi, P. Pahlevani, N. Bagheri, Y. Bendavid, and S. Rostampour, “Evaluating security pitfalls of ultra-lightweight IoT authentication: A critical

- analysis of permutation functions,” *Journal of Information Security and Applications*, vol. 93, p. 104188, 2025.
- [11] B. Tekin *et al.*, “Energy consumption of on-device machine learning models for iot intrusion detection,” *Internet of Things*, p. 100670, 2023.
- [12] C. Amuthadevi, S. P. R. Kavin, A. Shrestha, N. R. S. Yamini, and D. K. Sharma, “Tinyml-based intrusion detection system for sustainable iot security: A novel approach,” *Results in Engineering*, 2025.
- [13] A. Imteaj, U. Thakker, S. Wang, J. Li, and M. H. Amini, “A survey on federated learning for resource-constrained iot devices,” *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 1–24, 2022.
- [14] N. Albanbay, F. B. Ajaei, and A. Shami, “Federated learning-based intrusion detection in iot networks: Performance evaluation and data scaling study,” *Journal of Sensor and Actuator Networks*, vol. 14, no. 4, p. 78, 2025.
- [15] T.-D. Tran, P.-D. Bui, and V.-H. Pham, “EdgeTrust-Shard: Hierarchical blockchain architecture for federated learning in cross-chain IoT ecosystems,” *Journal of Systems Architecture*, vol. 173, p. 103701, 2026.
- [16] S. Huda, J. Yearwood, M. M. Hassan, and A. Almogren, “Securing the operations in SCADA-IoT platform based industrial control system using ensemble of deep belief networks,” *Applied Soft Computing*, vol. 71, pp. 66–77, 2018.
- [17] M. Mohamed and K. Alosman, “A Comprehensive Machine Learning Framework for Robust Security Management in Cloud-based Internet of Things Systems,” *Jurnal Kejuruteraan*, vol. 36, no. 3, pp. 1055–1065, 2024.
- [18] F. Heydari and Q. H. Mahmoud, “Implementation of lightweight machine learning-based intrusion detection system on iot devices of smart homes,” *Future Internet*, 2024.
- [19] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y. Arcas, “Communication-efficient learning of deep networks from decentralized data,” *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 54, pp. 1273–1282, 2017.
- [20] S. Hajj, J. Azar, J. Bou Abdo, J. Demerjian, C. Guyeux, A. Makhoul, and D. Gin hac, “Cross-layer federated learning for lightweight iot intrusion detection systems,” *Sensors*, vol. 23, no. 16, p. 7038, 2023.
- [21] N. Chaurasia, M. Ram, P. Verma, N. Mehta, and N. Bharot, “A federated learning approach to network intrusion detection using residual networks in industrial iot networks,” *The Journal of Supercomputing*, vol. 80, pp. 18 325–18 346, 2024.

- [22] F. Gusita, S. Rahimi *et al.*, “Lightweight cryptography for internet of things devices: a survey,” *Jurnal Ilmiah Teknik Elektro Komputer dan Informatika (JITEKI)*, vol. 11, no. 2, pp. 153–170, 2025.
- [23] B. Saravana Balaji, W. Paja, M. Antonijevic, C. Stoean, N. Bacanin, and M. Zivkovic, “IoT Integrated Edge Platform for Secure Industrial Application with Deep Learning,” *Human-centric Computing and Information Sciences*, vol. 13, p. 19, 2023.
- [24] P. Wang, L. T. Yang, J. Li, J. Chen, and S. Hu, “Data fusion in cyber-physical-social systems: State-of-the-art and perspectives,” *Information Fusion*, vol. 51, pp. 42–57, 2019.
- [25] Z. Alwaisi, E. Harjula, T. Kumar, and S. Soderi, “Securing constrained iot systems: A lightweight machine learning approach for anomaly detection and prevention,” *Internet of Things*, 2024.
- [26] S. Roy, J. Li, and Y. Bai, “Federated learning-based intrusion detection system for iot environments with locally adapted model,” in *2023 IEEE 10th International Conference on Cyber Security and Cloud Computing (CSCloud) / 2023 IEEE 9th International Conference on Edge Computing and Scalable Cloud (EdgeCom)*, 2023, pp. 203–209.
- [27] M. Amoon, T. Altameem, and A. Altameem, “RRAC: Role based reputed access control method for mitigating malicious impact in intelligent IoT platforms,” *Computer Communications*, vol. 151, pp. 238–246, 2020.
- [28] N. Moustafa, C. Schmitt *et al.*, “Ton.iot: The telemetry dataset for internet of things and industrial internet of things,” *IEEE Access*, 2020.
- [29] University of New South Wales (UNSW), “UNSW-NB15 Dataset,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- [30] —, “Bot-IoT Dataset,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/bot-iot-dataset>
- [31] —, “TON_IoT Datasets,” 2026, dataset download page. Accessed: 2026-02-07. [Online]. Available: <https://research.unsw.edu.au/projects/toniot-datasets>
- [32] M. A. Ferrag, L. Maglaras *et al.*, “Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning,” *IEEE Access*, 2022.
- [33] Stratosphere IPS, Czech Technical University in Prague, “IoT-23: A labeled dataset of IoT network traffic,” 2026, dataset page. Accessed: 2026-02-07. [Online]. Available: <https://www.stratosphereips.org/datasets-iot23>

-
- [34] E. C. P. Neto, J. P. Papa *et al.*, “Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.
 - [35] S. J. Rani, A. Sharma, and M. Zohaib, “Study for Integrating IoT-IDS Datasets: Machine and Deep Learning for Secure IoT Network System,” in *ACM Transactions on Sensor Networks (Article 3661167.3661286)*, 2024.
 - [36] K. Raminenei, V. Gupta, T. Durgam, and D. Kapila, “Development of a Machine Learning Model for Enhancing the Security of the Internet of Things (IoT) System,” in *Proceedings of the International Conference (Springer LNNS)*, 2023.
 - [37] National Institute of Standards and Technology (NIST), “NISTIR 8454: Lightweight Cryptography,” 2023, accessed: 2026-02-07. [Online]. Available: <https://csrc.nist.gov/pubs/ir/8454/final>
 - [38] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, “Federated optimization in heterogeneous networks,” in *Proceedings of Machine Learning and Systems (MLSys)*, vol. 2, 2020, pp. 429–450. [Online]. Available: <https://proceedings.mlsys.org/paper/2020/hash/38af86134b65d0f10fe33d30dd76442e-Abstract.html>

A. Anexos

A.1. Anexo A: Arquitectura del Sistema HFL

La arquitectura desplegada consta de tres capas principales y una extensión FOG opcional. La Figura A-1 resume el despliegue físico, los protocolos por enlace y la cobertura de ASCON-128 sobre cada canal.

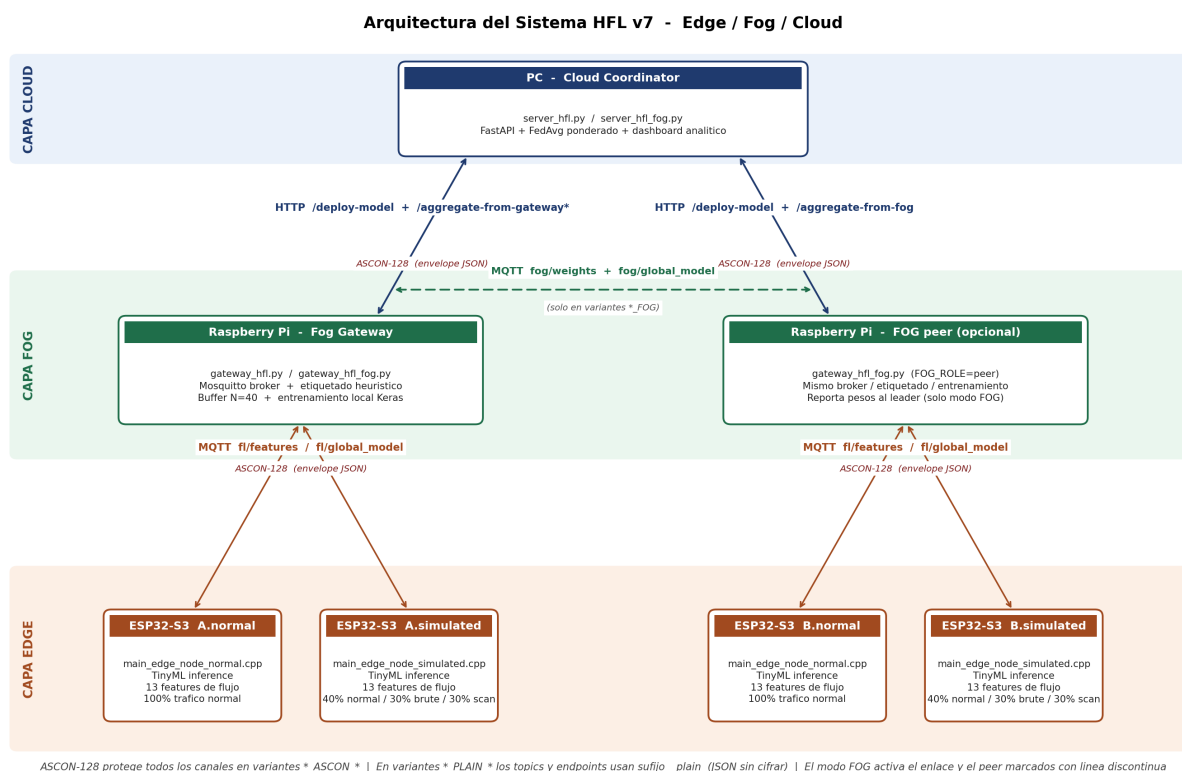


Figure A-1.. Arquitectura física del sistema HFL v7 sobre las capas Edge (ESP32-S3), Fog (Raspberry Pi) y Cloud (PC). La línea discontinua indica los enlaces `fog/*` activos solo en el modo FOG; los detalles por capa y la cobertura de ASCON-128 se describen en el texto.

A nivel narrativo, las capas se resumen así:

- **Capa Edge:** 2× ESP32-S3 por gateway (un nodo *normal* y un nodo *simulated*),

ejecutando inferencia TinyML y publicando `f1/features` hacia el gateway por MQTT cifrado con ASCON-128.

- **Capa Fog:** Raspberry Pi 4 actuando como broker MQTT (Mosquitto), gateway de entrenamiento local (etiquetado heurístico, buffer $N_s = 40$, entrenamiento Keras) y puente HTTP hacia el servidor.
- **Capa Cloud:** $1 \times$ PC ejecutando `server_hfl.py` o `server_hfl_fog.py` (FastAPI), responsable de la agregación FedAvg ponderada por número de muestras y del dashboard analítico en tiempo real.
- **Extensión FOG:** Raspberry Pi líder y peer comunicándose por `fog/weights` y `fog/global_model` para agregar entre gateways antes de reportar al PC mediante `/aggregate-from-fog`.

A.2. Anexo B: Archivos del Sistema

Table A-1.. Archivos principales del sistema HFL v7, agrupados por capa de despliegue.

Archivo	Dispositivo	Función
<i>Capa Edge (firmware C++ sobre ESP32-S3)</i>		
<code>main_edge_node_normal.cpp</code>	ESP32-S3	Nodo de tráfico normal: publica <code>f1/features</code> a ~ 5 s.
<code>main_edge_node_simulated.cpp</code>	ESP32-S3	Nodo de tráfico mixto (40 % normal, 30 % bruteforce, 30 % scan_A).
<code>ascon128.h</code>	ESP32-S3	Implementación nativa C++ de ASCON-128 (interoperable con Python).
<i>Capa Fog (Python sobre Raspberry Pi 4)</i>		
<code>gateway_hfl.py</code>	Raspberry Pi 4	Gateway federado en modo estándar (Mosquitto + entrenamiento local).
<code>gateway_hfl_fog.py</code>	Raspberry Pi 4	Gateway FOG con rol <code>leader</code> o <code>peer</code> (<code>fog/weights</code> , <code>fog/global_model</code>).
<i>Capas Fog y Cloud (utilidades comunes en Python)</i>		
<code>ascon128.py</code>	RPi / PC	Implementación Python de ASCON-128 con <code>generate_nonce(ts, ctr)</code> .
<code>ascon_metrics.py</code>	RPi / PC	Recolector de métricas de cifrado/descifrado a CSV.

continúa en la siguiente página

(continuación de la Tabla A-1)

Archivo	Dispositivo	Función
<code>plain.metrics.py</code>	RPi / PC	Equivalente para la línea base sin cifrado (variantes <code>*PLAIN*</code>).
<i>Capa Cloud (FastAPI sobre PC)</i>		
<code>server_hfl.py</code>	PC	Servidor central FedAvg + dashboard analítico.
<code>server_hfl_fog.py</code>	PC	Variante para agregación desde clusters FOG (<code>/aggregate-from-fog</code>).

A.3. Anexo C: Repositorio del Proyecto

El código fuente completo, datasets y scripts de entrenamiento están disponibles en el repositorio del proyecto. Las carpetas relevantes para esta revisión son `hfl_v7-RN` (MLP/RN con ASCON), `hfl_v7-no-ascon-RN` (baseline MLP/RN con JSON plano), `hfl_v7-CNN` (modelo CNN-1D con ASCON, estándar y FOG), `NotebooksAndModels` (entrenamiento offline y exportación de pesos), `Analisis de Modelos` (post-procesamiento por variante: RN, `no-ascon`, CNN, CNN_FOG, NIST) y `Papers` (literatura base).

A.4. Anexo D: Resumen de literatura clave incorporada

La Tabla A-2 resume los trabajos más recientes que sustentan las decisiones de diseño y los trabajos futuros del Capítulo 5, distinguiendo lo que aporta cada referencia.

Table A-2.. Literatura clave y aporte concreto a la tesis.

Trabajo	Tipo	Aporte a la tesis
Kaur <i>et al.</i> 2025 [9]	Survey ASCON (ACM CSUR)	Justifica elección de ASCON-128 sobre 128a y guía contramedidas SCA.
Mirzaali <i>et al.</i> 2025 [10]	Análisis crítico LWC	Argumenta no usar permutaciones <i>ad-hoc</i> ; sugiere ataques replicables.
Tran <i>et al.</i> 2026 [15]	FL Bizantino MCU	Encuadra brecha de memoria HFL-MCU y motiva trabajo futuro Bizantino.
Ficco <i>et al.</i> 2024 [6]	FL+TL on-board MCU	Antecedente directo de FL+TinyML on-board en ESP32 y RPi.
Dini <i>et al.</i> 2024 [5]	Survey IIoT/AI embed.	Encuadra <i>toolchains</i> TinyML y trade-offs SW/HW.

Trabajo	Tipo	Aporte a la tesis
Ni & Li 2024 [2]	Survey IIoT seguridad	Sustento del problema (IIoT, ML, retos).
Amanullah <i>et al.</i> 2020 [1]	Survey DL+IoT	Antecedente histórico de DL para seguridad IoT.
Bagaa <i>et al.</i> 2020 [4]	SDN+ML para IoT	Antecedente de framework ML para seguridad IoT.
Hajj <i>et al.</i> 2023 [20]	FL cross-layer IDS	Comparación directa contra FL para IDS IoT.
Chaurasia <i>et al.</i> 2024 [21]	FL+ResNet IIoT	Comparación con cliente FL más expresivo.
Roy <i>et al.</i> 2023 [26]	FL personalizado	Refuerzo del trabajo futuro de personalized FL.
Saravana Balaji <i>et al.</i> 2023 [23]	Edge sanitario+LABE	Antecedente de Edge+cifrado para IoT.
Wang <i>et al.</i> 2019 [24]	Data fusion CPSS	Sustento conceptual del modo FOG.
Rani <i>et al.</i> 2024 [35]	Integración datasets IoT-IDS	Sustento de la línea de trabajo futuro sobre datasets reales.